

LapSim engineering handoff

Mathematics, Python architecture, desktop interface, data provenance, verification, and known limits

Scope: current source-launched desktop application, its two simulation modes, their input data, and the code and checks needed to maintain them.

Audience: software lead and vehicle engineers.

Status: implementation description, not vehicle certification or a claim of predictive accuracy.

Source basis: this repository; the supplied *EV Vehicle Simulation and Torque Control* research report (Report I); and the supplied *EV Vehicle Simulation II: Aerodynamics, CFD, Track Conditions, and Secondary Effects* (Report II, 5 October 2026). Report II's proposals are treated as design evidence, not as proof that a feature exists or that its example values describe the team car.

Source of truth: docs/engineering_handoff.md

Contents

1. Read this first	4
1.1 Evidence levels	4
2. Run the code and locate the outputs	5
3. Repository architecture and execution boundaries	5
3.1 State ownership and side effects	6
4. Frames, signs, units, and terms	6
5. Data path and provenance	7
5.1 Course input	7
5.2 Car profiles and actual model values	9
5.3 Missing physical data	9
6. Main lap mode: aero, suspension, and tire capacity	9
6.1 Aerodynamic forces	9
6.2 Quasi-static suspension loads	10
6.3 Tire law and combined force	10
7. Main lap mode: powertrain, brakes, and battery	11
7.1 Propulsion and equivalent rotating mass	11
7.2 Hydraulic braking and regenerative allocation	11
7.3 RC battery and energy sign	11
8. Main lap mode: speed ceiling, controller, and cell update	12
8.1 Precompute the path speed envelope	12
8.2 Driver/controller action	12
8.3 Advance the vehicle by one distance cell	13

8.4 Optional experimental racing-line planning	14
9. What the main GUI displays	16
9.1 Live accepted-step view and completed playback	17
10. Four-wheel lab: state, control, tire, and motion equations	18
10.1 Power identity and integration	19
11. Report II upgrade: wind and road conditions	20
11.1 What Report II does and does not authorize	20
12. Four-wheel GUI inputs and outputs	21
13. Profile, source-data, and model audit trail	21
14. Saved records: exact schema, integrity, and replay limits	22
14.1 Main-lap record (`experiments/run_record.py`)	22
14.2 Programmatic one-lap model replay (`experiments/lap_replay.py`)	24
14.3 Four-wheel A/B record (`experiments/dynamics_record.py`)	24
14.4 Practical record inspection	25
15. How Python modules and the desktop interact	26
15.1 What to change for a new car profile	27
15.2 What to change for a new track	27
15.3 Numerical and code-review guardrails	27
16. Checks performed, interpretation, and known discrepancies	27
17. Prioritized engineering work and acceptance evidence	31

1. Read this first

LapSim has **two different calculation paths**. The main desktop's default route estimates one rolling-start lap along a prescribed centerline; its start and finish speeds are recorded and need not match at the closed-course seam. Its optional AI path comparison uses extra passes to close seam **speed** for each tested path. Both routes advance from one distance cell to the next with the repository's vehicle, powertrain, battery, brake, aero, tire, and quasi-static suspension models. Their outputs are model estimates for the selected car and course. The separate four-wheel lab integrates world position, body velocity, yaw, and four wheel speeds over time for a short, synthetic, open-loop torque-allocation maneuver. It is not an endurance-lap solver and does not use the selected Prius or TREV car profile.

That distinction is the most important interface contract in the project. A lap-time difference is not proof that the time-domain torque controller will be faster; a yaw difference in the lab is not a lap-time prediction. Neither mode currently has a calibrated 2026-27 car. The built-in Prius uses an equivalent electric power source to exercise the software, not a Toyota hybrid transaxle model. Optional TREV working profiles use partial local evidence and retain inherited defaults for missing subsystems.

The work informed by Report II adds explicit air-relative wind and per-wheel road-grip conditions to the four-wheel experiment. The **core API's** zero-drag, still-air, base-grip default recovers the pre-extension model; the desktop deliberately starts with a synthetic nonzero drag-area input. Full six-component CFD maps, ride-height feedback, vertical road/contact dynamics, and temperature-dependent limits remain future work because the required car, tire, CFD, road, and thermal data are not in this repository. Section 17 lists the exact data needed to advance those models.

An optional **experimental racing-line planner** now proposes one smooth path within an explicitly supplied corridor and computes model laps for it and a processed geometric centerline. A sampled integration of each prescribed curvature path must pass declared clearance and position-closure checks before either time can be ranked. The default fused course fails this gate, so its completed AI-mode times are **diagnostics, not a selected racing-line gain**. A separate synthetic rounded rectangle passes the software gates and demonstrates an eligible modeled selection under assumed widths; it is not a surveyed event track. The ordinary centerline remains the default driving mode and does not run optimization. The planner is deterministic geometry optimization rather than machine learning or closed-loop driving. The driver view plays solved telemetry on a reference map; it is not a measured or simulated tracked vehicle pose. Section 8.4 gives its math, Section 9 describes the desktop presentation, and `docs/ai_racer_design.md` records its research and evidence limits.

1.1 Evidence levels

Level	What it establishes here	What it does not establish
Equation and unit checks	Signs, conservation identities, limiting cases, numerical behavior of implemented equations	Parameters of the real car
Synthetic scenario check	Software responds to controlled changes in torque, wind, or grip	A measured lap-time or controller gain
Historical replay or fitted parameter	Agreement or disagreement with the recorded conditions and channels used	Independent prediction when parameters were tuned on the same run
Team-car validation	Would require held-out, synchronized measurements with configuration and uncertainty	Not currently available for the complete 2026-27 simulator

The older report's 44 reference checks belong to its separate source package. They were **not** tests of this repository. The repository's current checks are identified in Section 16.

2. Run the code and locate the outputs

The supported desktop launch is from the repository checkout on Windows:

```
cd C:\Users\lucas\Documents\Codex\LapTimeSim
.\setup_lapsim.cmd
.\launch_lapsim.cmd
```

The setup command finds 64-bit Python 3.11+, creates or reuses `.venv`, installs the project, and checks imports, the default course, Tk, and an actual TkAgg plot canvas. The launcher repeats that preflight, then starts a console-free window; an unexpected startup exception opens an error dialog and writes `%LOCALAPPDATA%\LapSim\logs\desktop_startup.log`. The direct `.venv\Scripts\python.exe -m lapsim.ui` route and `.vscode/launch.json` use the same checkout. All launch paths restrict numerical libraries to one worker thread and resolve Tcl/Tk scripts from the base Python installation when available. The application is native Tkinter with embedded Matplotlib; there is no distributable EXE. The default fused course loader expects checkout-relative `analysis/data/track/gnss_imu_endurance_track.csv`; packaging the Python module alone does not package this recording.

Run the repository checks with:

```
$env:OPENBLAS_NUM_THREADS='1'
$env:OMP_NUM_THREADS='1'
$env:MKL_NUM_THREADS='1'
.venv\Scripts\python.exe -m pytest -q
```

The environment lines limit numerical-library threads and memory demand on this computer; they are not physics parameters. GUI test runs here also set Tcl/Tk script paths explicitly when necessary. A passing suite establishes the stated software checks, not full vehicle accuracy.

This Markdown file is the editable report source. With ReportLab installed in a documentation Python environment, `python docs/render_engineering_handoff_pdf.py` regenerates `output/pdf/LapSim_Engineering_Handoff.pdf`; ReportLab is not required by the simulator itself.

Local user profiles are saved in `%LOCALAPPDATA%\LapSim\car_profiles.json`. Completed and interrupted main-lap runs are saved in `%LOCALAPPDATA%\LapSim\runs`; four-wheel A/B runs are saved in `%LOCALAPPDATA%\LapSim\dynamics_runs`. These files are outside Git; do not confuse them with the built-in or reviewed-source profiles. The status bar shows a shortened record ID; the file name contains the complete ID. Record contents are described in Section 14.

3. Repository architecture and execution boundaries

```
src/lapsim/ui/__main__.py
-> ui/app.py: Tk controls, worker thread, plots, profile comparison,
    optional racing-line workflow, driver playback, WIP tab
-> ui/garage.py and profiles/adapters.py: car construction and provenance
-> ui/simulation.py: course load/resample and default run_one_lap
-> optimization/racing_line.py: optional processed x/y baseline,
    full, half, bounded fourth path; up to eight
passes
-> solvers/path_constraints.py: cyclic speed ceilings
-> events/endurance.py: driver requests, braking, cell advance
    -> vehicle_model/vehicle.py: component coordination
        -> tire, suspension, aero, brake, drivetrain, battery
-> experiments/run_record.py: settings, results, aligned telemetry
-> ui/driver_view.py: time/distance interpolation onto reference x/y

-> ui/dynamics_lab.py: separate short maneuver and A/B displays
-> dynamics/planar.py: time-domain four-wheel forces and RK4
-> dynamics/conditions.py: deterministic wind and road queries
```

The environment is a pure model queried during integration, not a GUI animation. `vehicle_model/interfaces.py` defines replaceable component contracts. `lapsim` owns courses, solvers, controls, events, optimization, telemetry, profiles, and

experiment records. The `analysis/` tree contains source data, historical studies, preparation scripts, and archived material; its scripts do not run every time the desktop app starts.

`events/api.py::simulate_endurance` is a separate generic event/scoring interface. `solvers/speed_limit.py` and `solvers/lap_time.py` are other solver utilities. The desktop's actual one-lap route is `ui/simulation.py::run_one_lap` through `PathConstraintSolver` and `EnduranceSimulator`. Do not use a result from one API as if it documented the assumptions of another.

The generic prescribed-path scoring route now precomputes each cell's steady lateral-capacity speed with `PathConstraintSolver.local_corner_speed_limit_mps` and checks both entry and exit speed. A coarse curved cell can otherwise start with zero lateral force, gain speed during the cell, and receive skidpad points despite exiting above corner capacity. An excess beyond the 0.01 m/s numerical gate fails the event with its lap and cell identified and awards no points. This endpoint guard does not turn the cell-level force approximation into continuous tire-force or path-pose integration.

3.1 State ownership and side effects

The main `Vehicle` is mutable. `run_one_lap` resets it before a run. Component `update_state` methods commit accepted cell results; telemetry is recorded after accepted cells. The time-domain lab uses immutable state and control values for force evaluations. Its Runge-Kutta stages may call a force function repeatedly, so road and wind queries must be pure: an internal substep must not draw a new random weather value, heat a tire, drain a battery, or change a controller command. Report II Section 12 explicitly distinguishes force evaluation from committed state evolution.

The GUI is not the solver's clock. Lap work runs on a worker thread and returns results through a Tk queue. The four-wheel lab also computes away from the Tk event loop. Redrawing a plot or moving the time cursor must not change a run's physics or data.

4. Frames, signs, units, and terms

Symbol or channel	Meaning	Positive direction / units
<code>s</code> , <code>distance_m</code>	Distance along prescribed lap path	Increasing, m
<code>X</code> , <code>Y</code>	World-plane position in the four-wheel lab	World x/y, m
Body <code>x</code> , <code>y</code> , <code>z</code>	Vehicle axes	Forward, left, up
<code>psi</code> , <code>r</code>	Heading and yaw rate	Counterclockwise about +z, rad and rad/s
<code>u</code> , <code>v</code>	Body center-of-gravity velocity	Forward and left, m/s
<code>kappa</code>	Path curvature or wheel longitudinal slip, as named by context	Path: 1/m; tire slip: dimensionless
<code>Fx</code> , <code>Fy</code> , <code>Fz</code>	Tire force	Forward, left, upward reaction, N
<code>omega</code>	Wheel angular speed	Forward rolling positive, rad/s
<code>Tdrive</code> , <code>Tbrake</code>	Signed wheel drive and applied friction-brake torques	N m; brake torque opposes spin
<code>Ppack</code>	Battery terminal power in lap mode	Positive discharge, negative charge, W
<code>rho</code>	Air density	kg/m ³
<code>w</code>	Air velocity in world frame	m/s; distinct from car ground velocity

The four-wheel wheel order is **front-left, front-right, rear-left, rear-right**. Wheel body offsets use +y for the left side. A stronger right-rear forward tire force creates positive yaw because $M_z = x F_y - y F_x$ and the right wheel has negative y. The

lap solver uses a prescribed scalar path curvature; it does not integrate yaw dynamics with those independent wheel forces.

All public physics inputs are SI except fields that explicitly say rpm, psi, kWh, deg, or km/h. A dimensionless tire friction coefficient is not a force. The visible GUI converts speed from m/s to km/h, acceleration to g with 9.80665 m/s², forces to kN in traces, and battery power to kW. Those are display conversions; the core models use SI. The report's equations below use ASCII variable names so the exact code convention is clear.

5. Data path and provenance

5.1 Course input

analysis/data/track/gnss_imu_endurance_track.csv is the desktop's shipped 989 m course. The accompanying JSON says it is a spatial fusion of GNSS position, corrected IMU lateral acceleration, and an earlier schematic/registered centerline; it is not a surveyed current Formula SAE course. The metadata records 0.5 m source spacing, 1,979 points, a 0.76 m closure error, and limitations on map alignment and low-speed curvature. Some historical absolute file paths in that JSON describe the original preparation workstation; they are provenance strings, not valid paths on this computer.

src/lapsim/ui/course_catalog.py gives the desktop an explicit **Course** selector. team_endurance_fused_gnss_imu is the startup **Fused GNSS/IMU** · **default** choice and loads that CSV. synthetic_rounded_rectangle_v1 is **Synthetic loop** · **AI demo**, a generated 195.398224 m rounded rectangle made from two 40 m and two 20 m straights and four radius-12 m quarter-circle arcs. Its generated source cells are at most 0.5 m. This is an analytic software fixture with no measured cone positions or boundary widths, not a surveyed Formula SAE event map. Switching courses clears displayed results and replays; it does not switch the default **Centerline** driving mode to AI. The synthetic choice initially supplies editable *assumed* ±3 m AI half-width, 1.8 m vehicle width, and 0.2 m margin. These are scenario inputs, not sourced course data.

courses/course_bundle.py::CourseBundle adds a versioned import path for a team-prepared course. The user selects a UTF-8 JSON file with **Import course...**; the desktop validates and copies it to

%LOCALAPPDATA%\LapSim\courses\<bundle_sha256>.json, then reloads up to 64 saved JSON bundles at startup. A bad file is skipped with a warning. A course_id@revision collision with different manifest content in this local catalog is rejected; teams still need revision governance across computers. The bundled scripts/create_course_bundle.py converts a coherent SpatialTrack CSV to this format without source-code edits and hashes the exact CSV bytes. The converter checks that output does not already exist before saving and does not derive curvature from raw survey points.

docs/course_bundle_format.md defines every field and gives a working command.

V1 is deliberately narrow: 4–5,000 cells, at most 4 MiB of JSON, a declared right-handed local x/y frame in metres, one signed $\pm 2\pi$ lap, and piecewise constant curvature arcs whose stations/endpoints and closed heading agree within the numeric geometry gate. That gate uses **1e-6 m** per-cell and cumulative position tolerance, **1e-6 rad** heading-seam tolerance, and **1e-6 rad** agreement with the declared signed $\pm 2\pi$ turn. Frame handedness, origin, and axis descriptions are provenance declarations; the loader cannot compare them with a survey. It rejects duplicate JSON keys, nonfinite channels, wrong hashes, and malformed geometry. source_kind=measured is a **declaration with a source hash**, not independent survey verification. V1 requires boundary_status=absent and ai_defaults.width_source=assumed_uniform: neither the importer nor the AI planner can call its width a measured cone corridor. The inconsistent shipped fused recording fails this strict import contract and remains available only as its explicitly identified legacy built-in course.

SpatialTrack reads point arrays distance_m, x_m, y_m and one curvature value per cell. It checks finite values, channel lengths, and strictly increasing distance. It does **not** prove that the plotted x/y geometry exactly matches the cell curvature, that a closed flag closes the coordinates perfectly, or that the car remains within cones. The path solver consumes distance, cell length, and curvature. The plotted map consumes x/y. Vehicle.update_state separately integrates its achieved x/y/heading; the desktop does not overlay this path or check track width.

This difference is material to the optional planner. SpatialTrack.geometry_audit() now exposes a repeatable check: the shipped CSV has **989 m** of source station distance versus about **1,012.35 m** of accumulated x/y chords, about **0.760 m** endpoint gap, and **1,441 of 1,978 individual x/y chords longer than their assigned station intervals**, totaling 45.23 m of positive excess. A path segment cannot have a chord longer than its own travel distance. The largest absolute difference

between a prescribed constant-curvature arc's chord **length** and its plotted map-cell chord length is **0.235787 m**; this local diagnostic can expose defects that cancel in whole-lap turn and closure totals, although it does not check chord direction. The saved curvature integrates to **3.657937 rad** signed turn versus **6.283185 rad** of x/y winding. Starting at the first plotted chord tangent and integrating each constant-curvature cell as an arc leaves a **542.633 m** position-closure gap. This is a source-geometry diagnostic, not an integrated vehicle-lap pose. The Analysis tab displays a warning; the data channels are separately fused estimates rather than one self-consistent path. The experimental mode generates both its baseline and proposed line from the same prepared x/y geometry and recomputes curvature for both. However, even the processed baseline's closed polygon has a **0.750897 m** curvature-integrated closure gap because chord lengths and assigned arcs differ. Its time comparison is *not* relative to the ordinary 989 m source-curvature lap. The source metadata provides no surveyed left/right boundary, so a width entered in the desktop is a scenario assumption.

`ui/course_catalog.py::solver_track_for_course` chooses the standard centerline solver grid. For the fused recorded course it calls `ui/simulation.py::resample_track`, which creates cells at the requested maximum length. For source cells overlapping a new cell `[a,b]`, it computes

$$\text{kappa_new} = \text{sum}(\text{kappa_old_i} * \text{overlap_length_i}) / (b - a).$$

This conserves integrated signed curvature over the resampled path. It linearly interpolates x/y at the new distance boundaries. A new global grid can nevertheless average across original curvature changes, altering $\text{sum}(\text{kappa_i}^2 \text{ delta_s_i})$ and the tire demand at a given speed. Coarsening the fused course removes short curvature variation and can change lap time.

The synthetic course follows a different branch: for a requested maximum at least as large as its longest original cell (about 0.5 m), `solver_track_for_course` retains the generated source cells, so a 1 m or coarser request does **not** reduce its cell count or distort the exact circular arcs. For a finer request, it strictly validates agreement among the source stations, x/y endpoints, and prescribed constant-curvature cells, then subdivides each original cell into analytic straight or circular subarcs. Every source station and endpoint remains in the finer solver grid; interior x/y points follow the subarc rather than the old chord. The GUI defaults to a **requested maximum** 1 m step. The synthetic ID also requires the exact catalog-generated source. The geometry gate checks analytic endpoints cell by cell and the integrated closed heading; polygon chord winding is not an acceptance condition, since coarse valid arcs can alias it. This validation is a numerical solver-representation gate, not a survey or evidence of measured track boundaries.

Imported bundles take the same retain-or-analytic-subdivide branch. Their source geometry is checked when loaded and again before refinement; a coarser request keeps every original arc, and a finer request counts and splits each source cell without averaging adjacent curvature. The GUI's 5,000-cell compute cap applies to the resulting grid. The source bundle hash and solver grid hash are separate because subdivision changes the latter.

`solver_cell_count_for_course` predicts the grid before allocation and rejects a request for more than **5,000 actual generated centerline cells**. For the fused recording that count is $\text{ceil}(\text{course length} / \text{requested step})$. For a finer synthetic request it sums $\text{ceil}(\text{source cell length} / \text{requested step})$ over every original source cell; when the source grid is retained, its existing cell count is used. A quick total-length check can reject an obviously excessive request first, but the per-cell count is the final guard. The AI planner uses its own separate grid limit. Solver step is a physical/numerical modeling choice, not a plot-resolution option.

`SpatialTrack.refine(maximum_cell_length_m)` is a separate, unchanged on-demand resolution check. It subdivides each existing cell at most to the requested length while retaining every original station boundary, endpoint x/y, and piecewise-constant curvature. New x/y points lie on the old chord; this is **not** the synthetic course's analytic subarc subdivision. Thus each original cell keeps its length, signed turn $\text{kappa_i} \text{ delta_s_i}$, and squared-curvature integral $\text{kappa_i}^2 \text{ delta_s_i}$; splitting a cell does not average it with its neighbors. It rejects a nonfinite or nonpositive step and grids exceeding 100,000 cells. This QA method does **not** change the desktop's default discretization, certify geometric consistency, or repair an inconsistent source map. Speed-envelope and vehicle integration error can still change with cell size, so compare results rather than assuming convergence.

5.2 Car profiles and actual model values

`profiles/adapters.py::build_vehicle` constructs a **fresh** vehicle and a manifest. `repository_baseline` uses repository defaults. `prius_2026_1e` uses `ui/presets.py::make_prius_benchmark` and an idealized one-motor, front-drive power envelope based on published combined power. The Prius's mass and wheelbase are published benchmark specifications; its effective motor curve, pack, grip, and several aero/chassis inputs are assumptions. The saved-user profile mechanism (`ui/garage.py`) copies only seven editable Prius-equivalent fields: mass, peak-equivalent power, wheelbase, tire radius, tire friction, drag area, and speed limit. It is not a general editor for every Python model parameter.

The built-in Prius starter values are 1404.78 kg converted from 3097 lb, 144.67 kW converted from 194 hp combined power, 2.7508 m wheelbase converted from 108.3 in, and 0.3329 m **nominal geometric** radius from tire designation 195/60R17. Those four conversions are drawn from the manufacturer-specification block in `ui/presets.py`; geometric radius is not a measured loaded rolling radius. The editable starting assumptions are tire $\mu=0.95$, $C_dA=0.59 \text{ m}^2$, and top-speed cap 180 km/h. The constructed equivalent has a front driven axle, a four-knot idealized motor curve with a 4200 rpm knee and 10000 rpm maximum, unit motor/inverter/chain efficiencies, no charge-power allowance, zero aero lift, 60% static front weight, 0.53 m CG height, 1.56/1.58 m front/rear tracks, rolling-resistance coefficient 0.012, and cornering-drag coefficient 0.025. These are **software-model assumptions**, not a claim to reproduce a Toyota hybrid's mechanical or electrical architecture. Exact constructed values and overrides are frozen in each lap record.

Optional TREV5 working profiles require the reviewed ENME408 data package in Downloads. `profiles/registry.py` validates the package and `profiles/adapters.py` promotes only allowlisted, unit-compatible fields. The manifest separates selected source values, inherited model defaults, unused records, conversion details, source locations, artifact hashes, and limitations. A source browser can inspect records without silently applying them. These profiles are partial working scenarios; neither motor count nor 2026-27 electrical topology should be inferred from a label.

The effective vehicle snapshot in each saved lap record is the **constructed car used by that run**, not merely the chosen base profile. Editable overrides are recorded separately. This matters because a user can select Prius, change a number, run once, and leave the named profile unchanged.

5.3 Missing physical data

The shipped course has no track width, elevation, bank, crest radius, local surface material, wetness, roughness, road temperature, or measured wind. The main lap mode therefore assumes a flat, uniform, still-air path with its configured constant density. A road patch drawn in the four-wheel lab is a deterministic synthetic sensitivity input, not a reconstructed patch from this course. No rain graphic is treated as tire physics.

6. Main lap mode: aero, suspension, and tire capacity

The lap model is quasi-static within each spatial cell. It does not solve a Navier-Stokes fluid field, suspension differential equations, or four independent wheel angular speeds. Its components provide algebraic forces to the path solver and the vehicle's cell update.

6.1 Aerodynamic forces

`vehicle_model/aero/model.py::Aero.forces_n` computes dynamic pressure $q = 0.5 \rho v^2$, positive drag $D = q C_d A$, and signed downforce $D_{\text{down}} = -q C_l A f_{\text{roll}}$. $C_l < 0$ means downforce under this model's convention. The front share is $p D_{\text{down}}$; the rear gets $(1-p) D_{\text{down}}$. The roll factor is

$$f_{\text{roll}} = 1 - \min(\text{abs}(\text{roll})/\text{roll_limit}, 1) * (1 - \text{retention_at_limit}).$$

At and above the default one-degree roll limit, the default retains 50% of the zero-roll downforce. This is a configured approximation, not a measured ride-height/attitude map. `Vehicle.aero_forces_n` obtains a quasi-static body-roll estimate from suspension before calling `Aero.forces_n`. The optional `ActiveAero` subclass selects a two-position low-drag state from absolute path curvature and scales drag and downforce independently; it has no actuator delay or rule-eligibility determination. The desktop's default Prius uses its own estimated C_dA , not the baseline aero package's coefficient table.

This lap calculation currently uses **ground speed as airspeed** and a fixed air density. It does not include crosswind side force, aero yaw/pitch moments, or the wind-aware model in the separate lab. Its baseline coefficient and reference area values are listed in `docs/model_parameters.md`. The active ratio $\text{abs}(C_l)/C_d$ must be computed from those actual coefficients; a historical unused ratio constant was removed because it disagreed with them.

The total normal reaction used by resistance and tire calculations is approximately $m g + D_{\text{down}}$ on the flat path. A positive-lift configuration that removes all reaction or makes either axle's base reaction negative is now rejected as outside this constrained-contact model. It is not silently turned into extra tire grip or a negative denominator. The code still cannot simulate flight, porpoising, or a crest.

6.2 Quasi-static suspension loads

`vehicle_model/mech/suspension.py::tire_normal_loads_n` starts from static axle weights and adds the configured front/rear aero downforce. Longitudinal load transfer is proportional to $m a_x h_{CG} / L$, moving load rearward under forward acceleration. Lateral transfer uses $F_y = m a_y$, front/rear roll-axis heights, track widths, total elastic roll stiffness, and the configured front load-transfer distribution. It estimates body roll for the aero multiplier. Wheel loads are clipped at wheel lift only within its algebraic model; it has no heave, pitch, roll, damper, unsprung-mass, or contact-trajectory states. Do not add a second independent load-transfer correction around this component.

More specifically, its quasi-static roll angle is $\phi = m a_y \max(h_{CG} - h_{\text{roll_axis_at_CG}}, 0) / K_{\text{roll}}$. Geometric roll moments act through front/rear roll-axis heights; elastic roll moment is distributed according to a front stiffness fraction chosen to approximate the configured front lateral-load-transfer fraction within $[0, 1]$. Each axle's lateral transfer is its geometric plus elastic moment divided by that axle's track, then limited so no algebraic wheel load is negative. The exact stiffness-fraction solve is in `suspension.py::tire_normal_loads_n` and is included in the effective vehicle snapshot; it is an equilibrium allocation, not a transient suspension response.

Report II gives a more complete flat-road pitch balance including aerodynamic pitch moment and longitudinal ground force. The present model is **not** an exact solution of that general balance: it has no aerodynamic pitch moment, wind, grade, or transient pitch angular momentum. Its outputs are suitable for model sensitivity and force-budget checks under the stated flat, quasi-static assumptions.

6.3 Tire law and combined force

`vehicle_model/mech/tire.py::Tire` is the default main-lap tire. It interpolates a load-sensitive $\mu(F_z)$ table over its configured range and clamps beyond the table. Pure lateral and longitudinal capacities are $\mu_{\text{lat}}(F_z) \max(F_z, 0)$ and $\mu_{\text{long}}(F_z) \max(F_z, 0)$. It distributes a requested total lateral force across wheels in proportion to capacity. Its remaining longitudinal limit under lateral force is a friction-circle-style capacity:

$$F_{x_{\text{remaining}}} = F_{x_{\text{pure}}} * \sqrt{\max(0, 1 - (F_y / F_{y_{\text{pure}}})^2)}.$$

Drive demand is allocated only to configured driven wheels; braking requests are distributed by axle and constrained at each contact patch. The default main-lap tire does not integrate each wheel's $J d(\omega)/dt$. It infers a longitudinal slip estimate from force/capacity, with a configured peak-slip ratio and optional distance relaxation. Tire saturation and the path speed ceiling are therefore not a substitute for a measured force-versus-slip-and-temperature map.

Optional Pacejka classes and a public UPC `.tir` file exist in `vehicle_model/mech/pacejka.py` and `data/tires`. They are **not** silently enabled by the desktop. The imported file's pure lateral information and its longitudinal/combined-slip provenance differ; see `docs/model_parameters.md` before using them as a measured complete tire model.

7. Main lap mode: powertrain, brakes, and battery

7.1 Propulsion and equivalent rotating mass

The baseline power chain is DC pack terminal -> inverter -> motor shaft -> chain/final drive -> driven tires. `motor.py` interpolates peak torque against RPM and limits shaft power and maximum RPM. `drivetrain.py` uses a nominal no-slip conversion from ground speed through the tire rolling radius and gear ratio for its envelope; cell force/power checks can then use inferred driven-wheel surface speed from tire slip:

```
omega_wheel = v / R; omega_motor = gear_ratio * omega_wheel;
Fx_wheel = Tmotor * gear_ratio * eta_chain / R.
```

The available motor torque also depends on pack discharge power, inverter and motor efficiencies, and motor power limit. `vehicle_model/vehicle.py` can further reduce demand when the driven wheel surface speed would violate the motor envelope or configured vehicle speed cap. The default efficiencies are constants, so there is no torque-versus-temperature derating or detailed inverter loss map. The continuous motor curves are available as references/telemetry; the one-lap event does not thermally integrate continuous rating usage.

Main-lap wheel, shaft, and rotor inertias are **reflected into an equivalent longitudinal mass** $m_{\text{eff}} = m + J_{\text{ref}}/R^2$, where $J_{\text{ref}} = (J_{\text{motor_rotor}} + J_{\text{chain_input}}) \text{ratio}^2 + J_{\text{chain_output}} + J_{\text{driven_wheels}}$. This belongs only to the distance-domain car, which does not integrate wheel speed as independent state. The four-wheel time-domain model instead integrates four $J d(\omega)/dt$ equations and must not also add the same wheel inertia to m . This separation prevents double counting.

The built-in repository defaults include parameters that were fit to the historical endurance run: rolling radius, motor efficiency, and chain efficiency. Agreement with that same run is in-sample consistency. It cannot by itself validate a prediction for a future car, another tire, or another track.

7.2 Hydraulic braking and regenerative allocation

`vehicle_model/mech/brakes.py` converts independent front/rear pressure requests, after its configured deadband/map, to axle torque and then force through rolling radius; both pressures have a 300 psi baseline ceiling. Tire force is the final cap. The endurance automatic path controller asks for enough negative force to reach the next speed ceiling and distributes friction brake requests across the two axles and their left/right tires. The pressure inverse is an actuator request, not a guarantee that the path is feasible.

`Vehicle.maximum_regenerative_brake_force_n` limits a requested regen magnitude by motor speed/torque, drivetrain efficiency, and pack charge power. `Vehicle.update_state` checks driven-axle topology, and `Tire.calculate_forces` then limits the combined regen and friction-brake request by contact grip. `EnduranceSimulator` enables automatic regen only if a non-null SOC threshold is configured and SOC is below that threshold. The desktop `EnduranceRunConfig` does **not** set this threshold, and the baseline/Prius default pack charge-power cap is zero. Thus a visible regenerative-braking trace may correctly stay at zero. Friction-brake heat is not modeled, and regenerative energy is signed battery power rather than disc heating.

7.3 RC battery and energy sign

The repository default is a one-RC Thevenin pack in `vehicle_model/electrical/battery.py`; the Prius benchmark creates an `OCVPackBattery` equivalent. Both use pack open-circuit voltage versus SOC and a terminal resistance. For the RC variant, with positive current discharging:

```
V_source = OCV(SOC) - V_polarization;
V_terminal = V_source - I R0;
P_terminal = V_terminal I;
I = 2 P_terminal / (V_source + sqrt(V_source^2 - 4 R0 P_terminal));
SOC_next = clip(SOC - I dt / (3600 capacity_Ah), 0, 1);
V_polarization,next = exp(-dt/(R1 C1)) V_polarization
+ R1 (1 - exp(-dt/(R1 C1))) I.
```

The low-current quadratic root avoids cancellation near zero power. Pack limits also bound terminal voltage, discharge/charge power, SOC, and the quadratic branch. A positive `battery.power_w` means discharge; negative means charging. The event's displayed **net energy** integrates signed terminal power over accepted cells and converts joules to kWh. The telemetry recorder also has a separate cumulative **positive-discharge** energy channel; these must not be interchanged. The pack has no cell temperature, cooling, aging, or measured Prius hybrid energy flow. `docs/battery_model.md` and `docs/battery_rc_validation.md` describe the model and the limited historical voltage holdout.

8. Main lap mode: speed ceiling, controller, and cell update

8.1 Precompute the path speed envelope

`ui/simulation.py` builds a constant-fraction, two-knot periodic torque profile; both knots have the same requested fraction, so this desktop input is a constant normalized driver demand. It creates `PathConstraintSolver` with explicit tolerance/pass/iteration settings from `path_solver_settings`. At each curvature cell the solver bisects speed between zero and the configured maximum until required lateral force $m v^2 \text{abs}(\kappa)$ fits the sum of quasi-static tire lateral capacities. The local limit is then passed through cyclic backward braking passes, using the next cell's speed and available braking force to bound each entry speed. Conceptually:

$$v_{\text{entry}}^2 + 2 a_{\text{brake}}(v_{\text{entry}}, \kappa, \text{car}) \Delta s \leq v_{\text{next}}^2.$$

The implemented braking acceleration is nonlinear in speed and load transfer, so the solver evaluates force balance and bisects rather than assuming one constant deceleration for the entire lap. It iterates around the closed path until its configured convergence tolerance or pass bound is reached. `PathSpeedConstraints` stores local corner limits and braking ceilings for every cell. A nonzero speed ceiling is a model output, not a safety guarantee for a real car.

8.2 Driver/controller action

`events/endurance.py::EnduranceSimulator.run` resets the vehicle and, unless an explicit start speed was supplied, begins at the **first cyclic braking ceiling**. This is a rolling-start, single initial-condition lap, not a standing-start acceleration event or a converged periodic flying lap. The cyclic ceiling guarantees braking feasibility, not that the car can accelerate back to its entry speed at the seam. The result records actual entry and exit speeds and exposes finish-minus-start speed for a completed single lap. Before each cell it checks entry against local and braking ceilings. The automatic torque-profile controller converts driver fraction to requested motor torque, computes the force needed to reach the next ceiling, and chooses one of drive, coast/torque reduction, or hydraulic braking (with optional configured regen). Steering is chosen from the prescribed curvature with $\delta = \text{atan}(\kappa L)$. A full `EnduranceControlProfile` can instead supply explicit torque, pressure, regen, and steering; that path is checked against speed and curvature constraints rather than silently repaired. `Vehicle.update_state` derives requested curvature from steering and may reduce achieved curvature when lateral tire force saturates. After each model step, the event requires **both** requested and achieved curvature to differ from the prescribed cell curvature by no more than `EnduranceRunConfig.path_curvature_tolerance_per_m` (**1e-9 1/m** by default) before accepting the cell. Thus a direct zero-steer run on a circular path fails. This checks a scalar curvature for each cell, not whether the integrated x/y pose follows the plotted course or remains within cones. The automatic baseline and AI laps request the prescribed curvature, but they also lack a closed-loop pose and boundary check.

The ordinary single-car lap uses that first cyclic ceiling. The two-car A/B workflow instead computes one common feasible rolling start from both cars' first ceilings, then supplies it explicitly to each event; Section 9 gives the formula and record contract. Neither route requires the exit speed to equal its entry speed.

The automatic driver targets the smaller of the **next cell's braking ceiling** and the **current cell's local corner-speed ceiling**:

$$v_{\text{target}} = \min(v_{\text{braking,next}}, v_{\text{corner,current}}).$$

The current-cell cap matters when a corner is followed by a straighter cell: following only the next ceiling could accelerate above the corner limit before leaving the curved cell. After the model step, the event checks the requested and achieved cell

curvature as above, then $v_{\text{exit}} \leq v_{\text{corner,current}} + \text{epsilon}$ and $v_{\text{exit}} \leq v_{\text{braking,next}} + \text{epsilon}$, with $\text{epsilon} = \text{config.path_speed_tolerance_mps}$ (**0.01 m/s** by default). These checks apply to supplied explicit controls as well as automatic control. The event records telemetry and progress only for an accepted cell. Because the prescribed curvature and longitudinal acceleration are constant over a cell, bounding its entry and exit speeds bounds the modeled speed between them. The cell model still evaluates forces at a representative operating point and has no transient tire/path tracking; these are consistency guards within that approximation, not real-world cornering validation.

The automatic brake allocator inverts an estimated axle-force request into pressures. When either axle reaches the effective configured pressure cap, the controller now commands both axles at that cap, matching the full-pressure premise of the backward braking envelope. The committed vehicle load-transfer solve can otherwise deliver slightly less force than the inverse estimate; small deficits accumulated into an apparent ceiling failure in one TREV5 working scenario. The tire combined-slip model still caps contact-patch force, and pressure never exceeds the configured or physical limit. This is a conservative simulation guard, not a calibrated brake-bias or ABS controller; a prospective pressure solve against the actual vehicle step remains future work.

If the vehicle exceeds a ceiling, stops, exceeds time, depletes SOC, or cannot traverse a cell, `EnduranceRunResult` carries a failure reason and completed-lap count. A failed comparison is not ranked as a valid lap. The desktop saves its record for diagnosis. `Vehicle.update_state` can advance a cell before a post-step rejection, so the failed result's existing terminal time/speed/SOC may describe that attempted cell. The added `accepted_*` fields identify the checked prefix that agrees with energy, telemetry, and Driver progress. A cell that completes physically and depletes SOC at its endpoint is included in those checked outputs before the run stops. Section 14.1 gives the exact saved fields and remaining rollback limitation.

The separate legacy `solvers/lap_time.py::LapTimeSolver` forward acceleration pass now also caps a cell's exit speed by both the current and next `SpeedLimitMap` limits, as well as reachable speed. This applies the same current-cell corner principle to that lower-level speed-profile API; it does not turn its profile into an `EnduranceSimulator` lap or add path-pose validation.

8.3 Advance the vehicle by one distance cell

`Vehicle.update_state(controls, delta_s)` obtains tire, brake, motor, aero, and suspension forces and solves a longitudinal force-balance fixed point. Required lateral force is $m v_{\text{entry}}^2 \text{abs}(\kappa_{\text{request}})$. If tire capacity is lower, a root solve reduces achieved lateral force and therefore effective curvature. Resistance is

```
F_resist = D_aero + Crr (m g + Ddown)
          + Ccorner Fy^2 / (m g + Ddown).
a_x = (Fx_tire - F_resist) / m_eff.
```

The normal-load guard rejects unsupported negative reaction before the denominator is used. The cell kinematics assume constant longitudinal acceleration at the cell entry operating point:

```
v_exit^2 = v_entry^2 + 2 a_x delta_s;
dt = 2 delta_s / (v_entry + v_exit).
```

Time and force are coupled because optional tire slip relaxation may depend on dt . The code iterates to a relative time tolerance and refuses a cell it cannot traverse. It then commits component states using the converged dt , integrates effective heading by $\text{delta_heading} = \kappa_{\text{effective}} \text{delta_s}$, and adds the corresponding circular-arc x/y increment. This is a spatial, quasi-static approximation. It does not integrate lateral velocity, yaw inertia, road banking, or track containment.

8.4 Optional experimental racing-line planning

`src/lapsim/optimization/racing_line.py` prepares a **new path input** for the unchanged lap model. It does not change the force, power, braking, or battery equations in Sections 6-8. The desktop imports this module only after **AI racing line (experimental)** is selected. Ordinary **Centerline (default)** calls `solver_track_for_course` and `run_one_lap` on the selected course's original distance/curvature channel and pays no optimization cost. That grid helper resamples the fused recording or preserves and analytically subdivides the synthetic source's existing straight/arc cells as described in Section 5.1. The startup course is the fused team recording; the synthetic course is selected separately. The AI route requires the user to enter assumed half-width, vehicle width, and safety margin. Its programmatic `TrackCorridor` supports separate left/right width for *each source cell*, plus a required source description; the current desktop supplies equal uniform assumed widths because there is no surveyed corridor. The clearances must exceed $\text{vehicle_width}/2 + \text{margin}$ on both sides. These values define a center-of-car offset envelope, not an observed edge of pavement.

For a closed source course with length L_{source} , the planner removes the x/y endpoint mismatch linearly over source distance, interpolates x/y with a periodic cubic spline onto $N = \max(\text{ceil}(L_{\text{source}}/2 \text{ m}), 4 K, 32)$ periodic samples by default ($K=24$ controls), and applies wrapped Gaussian smoothing with a 0.8 m scale. Periodic cubic interpolation avoids artificial corners when the source and planner station grids differ; it does not repair an inconsistent source map. The planner records closure error, maximum preparation shift, and processed path length difference. The processed zero-offset line is the **geometric baseline**; the ordinary source-curvature line is not a valid A/B baseline for a shifted x/y candidate. The planner refuses more than 5,000 samples, an open course, self-intersecting processed baseline, undefined tangent, degenerate cell, or unusable corridor.

At each processed sample, a central-difference tangent defines the unit left normal n_i . A periodic uniform cubic B-spline basis B_{ij} maps the K controls c_j to lateral offset $d_i = \sum_j B_{ij} c_j$; candidate sample positions are $p_i = p_{\text{base},i} + d_i n_i$. Because the basis is periodic, its offset and first derivatives meet at the lap seam. The algorithm computes candidate chord lengths $\text{ell}_i = |p_{i+1} - p_i|$, cumulative solver station $s_{i+1} = s_i + \text{ell}_i$, and signed heading turn θ_i from the incoming and outgoing chord vectors a and b at each vertex:

```
theta_i = atan2(cross(a,b), dot(a,b));
kappa_cell,i = (theta_i + theta_{i+1}) / (2 ell_i);
sum_i (kappa_cell,i ell_i) = sum_i theta_i = 2 pi * winding_number;
J(c) = sum_i [kappa_cell,i^2 ell_i] / sum_i ell_i
      + lambda (sum_i ell_i / L_baseline - 1), lambda = 0.01 m^-2.
```

The discrete curvature is a centered allocation of polygon heading turn to neighboring solver cells. Its integral matches the polygon's signed winding exactly, but the local value remains a numerical approximation for the speed-envelope solver.

Matching total turn does not make a constant-curvature arc at the assigned cell length reach the polygon chord endpoint. It is recomputed for **both** geometric baseline and proposed path, never copied from the original source curvature. The first term of J has units m^{-2} ; the length-penalty coefficient therefore also has units m^{-2} , since the relative-length term is dimensionless. J favors lower average squared curvature with a weighted length penalty. It is **not** lap time, and it does not use car mass, power, grip, or aero while solving. SciPy SLSQP starts from all-zero offset, uses finite-difference objective derivatives, and is capped at 60 iterations by default. The numeric objective is multiplied by 10,000 for the optimizer's absolute tolerances; reported objective values remain unscaled. This is one local candidate, not a global racing-line optimum or a trained policy.

The linear optimizer constraints evaluate the spline at planner nodes and midpoints and at every original source-cell boundary and midpoint. Bounds at shared boundaries use the narrower adjacent width, including the closed seam's first and last cells. A postcheck analytically finds the cubic offset extrema *within each source cell* against its piecewise constant left/right widths. Before planning, a normal-coordinate fold guard rejects a supplied inside clearance whose local curvature-times-offset reaches 0.98 , since the offset strip approaches its local bend radius and cannot support a meaningful corridor claim. The candidate must have violation no more than $1\text{e-}6 \text{ m}$, forward local motion relative to the processed baseline, finite positive chord lengths, and no nonadjacent segment intersection. The latter check operates on the discretized polygon. Width certification at this stage is for the mathematical **offset polygon relative to the processed reference and supplied cell widths**, not the arc path actually implied by the lap model.

The CurvaturePathAudit separately integrates the saved constant-curvature cells as the model prescribes, starting at the processed path's first x/y point with an initial heading inferred from its first chord and start turn. Its sample set is the union of **four quarter-cell positions per modeled cell, every source-corridor cell boundary, and every source-corridor cell midpoint**. Duplicate stations are removed. This catches a narrow supplied source cell that could fall entirely between two quarter-cell samples. At a shared source boundary the usable left/right widths are the narrower of neighboring cells; station L wraps to station zero, so the closed seam uses the narrower first/last widths. Each sample is compared with the declared usable normal-coordinate corridor around the processed baseline. For an eligible path, sampled excess must be at most **1e-8 m**, and the integrated end position must close to within **0.01 m** of the start. This is a deliberately conservative screening rule for the supplied assumed corridor; it is not a continuous swept-body or surveyed-pavement/cone certificate. It does not replace a consistent track survey or pose tracking. The original shipped processed baseline fails, so a completed physics run can have time and telemetry while being ineligible for an AI line ranking.

The audit also reports `minimum_corridor_slack_m`, the least remaining usable clearance at those sampled positions. The new fourth-trial search requires at least **0.02 m** of extra sampled slack beyond the user-entered width and safety margin. This is a selection buffer for coarse sampling; it does not certify continuous clearance or represent measurement uncertainty.

`compare_lines_with_lap_model` copies the selected vehicle separately for the geometric centerline and every tested candidate, preventing one lap's battery or chassis state from contaminating another. All use the same torque fraction and cell physics. It first tests geometric baseline strength 0, full offset 1, and half offset 0.5. If all three audits and times are eligible and half beats both endpoints by more than 0.05 s, a convex quadratic through their times chooses one safeguarded fourth strength from {0.25, 0.375, 0.625, 0.75, 0.875}. If full fails its sampled audit while eligible half beats baseline by more than the selection margin, a geometry-only preflight checks {0.975, 0.95, 0.9, 0.875, 0.75, 0.625} in descending order. The first valid path with at least 0.02 m additional sampled slack gets the fourth model trial. Otherwise the fourth strength is 0.75. The quadratic and clearance screen choose only where to run that trial; neither establishes its eligible time. Convex offset scaling preserves the supplied lateral bounds for the proposed **polygon**, while each intermediate path rebuilds station and curvature and checks direction and nonadjacent crossings.

The desktop calls the speed-only seam helper for at most four paths. It solves path constraints once per path, starts a dry probe at the cyclic braking ceiling, then makes a recorded pass at that probe's exit speed. Each pass uses a fresh copy of the same initial car and pack; only the final pass emits progress and telemetry. At most **eight complete lap-model passes** occur. A comparison time requires a completed final pass with $\text{abs}(\text{finish_speed} - \text{start_speed}) \leq 0.005 \text{ m/s}$ and a passing modeled-path audit. Speed closure at fixed initial vehicle state is **not** periodic battery charge or full-state convergence. Completed but ineligible times remain diagnostic with their run and track. If the processed baseline audit fails, status is `invalid_processed_baseline` and no path can win. Each completed AI path record saves its explicit start speed. An eligible candidate wins only if its gain over eligible baseline is **strictly greater than 0.05 s**. A smaller positive gain is `unresolved_close_gain`: the baseline stays selected and the candidate remains inspectable. The 0.05 s margin is a provisional heuristic based on observed grid sensitivity, **not** a certified error bound. A finer-grid study is needed to rank close paths. If only a candidate has an eligible time for a reason other than failed baseline path audit, it may be displayed without claiming a gain. An optimizer stop without success can still supply a geometrically feasible candidate; a lower geometry objective J can still lose the car-dependent time comparison. This is a coarse car-dependent line-strength search, not arbitrary-path minimum-time optimization. The separate wind/road lab does **not** feed this planner or the main flat, uniform, still-air lap.

An unmerged **in-memory biarc prototype** explored connecting the intended x/y endpoints with curvature-consistent arc segments. It reduced geometric mismatch to roughly **1e-9 m**, but took **2.2–2.5×** as long and changed modeled lap times by **5–7 s** because of curvature spikes. A separate unmerged one-circular-arc-per-edge experiment used an odd **991-cell** path and closed its integrated geometry to about **2e-11 m**, but increased peak absolute curvature and adjacent curvature jumps. Its one-pass times shifted by **5–7 s**, one TREV numerical ranking reversed, and compute rose about **2.5×**. Exact position closure alone does not establish physical or numerical validity. Neither prototype is used by the simulator; any replacement needs consistent geometry, controlled curvature, course review, and independent resolution and vehicle checks.

Research basis: TUM FTM's trajectory planner and the TORCS robot tutorial. LapSim's code is original; `docs/ai_racer_design.md` has source links and geometry checks.

9. What the main GUI displays

View/control	Source and meaning
Course	Explicit fused GNSS/IMU default, synthetic AI demo, or validated versioned import; changes input course and clears prior results/replay without enabling AI
Car profile / editable boxes	Fresh vehicle from <code>build_vehicle</code> or editable Prius-equivalent <code>VehicleSetup</code> ; seven car fields editable on Prius and saved copies
Driver request	Constant fraction, 0-100%, of motor torque-curve request before vehicle/path limits
Max solver step	Requested maximum centerline cell length in metres; fused source is resampled, while synthetic and imported coherent sources retain analytic cells for coarse requests and subdivide source arcs for finer requests; AI mode independently builds a nominal 2 m geometric grid
Driving path	Centerline is default; experimental AI requires assumed corridor half-width, vehicle width, and safety margin
Run one lap	Worker executes one rolling-start prescribed-path lap in default mode; AI mode plans one geometry candidate and tests full, half, and one bounded fourth offset against its processed centerline
Compare car profiles	Two saved/built-in profiles run against the same course, step, driver fraction, and feasible rolling-start speed; B-minus-A numbers and separate finish/seam speeds are model differences
Eight boxes	Lap time, peak speed, average speed (<code>course_length/time</code>), course distance, signed net pack-model energy, peak <code>abs(lateral acceleration)/g</code> , and entry/exit speeds
AI result boxes	Eligible baseline/candidate times and signed difference when both audits pass; completed audit-failed times carry *, difference is blank, Compare path numbers is disabled, and text gives sampled excess and seam gap
Trace selector	Speed, acceleration, drive/braking forces, inferred driven slip, or battery power from named telemetry channels, against time or distance
Course map	Selected source x/y reference, fixed top-down; mouse drag pans and wheel zooms; the fused course displays its geometry warning while the synthetic option is labeled as an assumed example
Driver view tab	Latest accepted physics cell during a solve, then timed top-down replay on each run's saved solver-grid x/y with fixed car marker, play/pause, scrub, rate, wheel zoom, cell control/force number boxes, and a Replay lap menu for completed comparisons
Timed sessions · WIP tab	Scope text and disabled start button for the future ghost/controller/complete-session/replay-tolerance workflow
Source data / model notes	Inspect reviewed records, origins, model use, and caveats; inspection does not change the active vehicle
Dark mode	Reverses white/black Tk and Matplotlib surfaces; it does not change a simulation parameter

The GUI deliberately keeps numeric output boxes and simple monochrome traces. Car and path comparison windows name the selected course, and that label remains visible if the main window later switches courses. A solid line means A/current run and a dashed line means B in comparisons. Do not infer measurement uncertainty from line thickness, and do not interpret a visually overlapping trace as an identical force or energy history. For exact values and full channels use the saved JSON record.

For A/B, `prepare_one_lap_constraints` solves each car's limits once on the same grid. Its wrapper binds the limits to that vehicle; reuse with another car or track is rejected. The app starts both at `min(ceiling_A[0], ceiling_B[0])` m/s. Keep each car's setup unchanged between limit preparation and running.

Both cars then complete one recorded pass with that explicit rolling start and the same constant driver torque fraction. The constraints are reused instead of solved again; the extra shared-start selection adds no lap-model pass. Both records freeze the same start speed in their endurance settings, so `replay_lap_record()` can reproduce each saved command trace. Each car still has its own tire/powertrain limits, battery state, achieved finish speed, and nonperiodic speed seam. The B-minus-A lap difference is a same-start **model sensitivity comparison**, not a steady-state race prediction or validation against either car.

9.1 Live accepted-step view and completed playback

`EnduranceSimulator.run` accepts an optional `progress_callback`. After a cell passes the path, speed, traversal, time, and stall checks, it emits an immutable `LapProgressSnapshot` with zero-based lap/cell indices, cell count, elapsed model time, current-lap station, total model distance, speed, and lateral acceleration. A cell rejected by those checks emits no snapshot. A cell that reaches zero pack SOC at its endpoint is present in both telemetry and progress before the event stops with a depletion failure. `ui/simulation.py::run_one_lap` forwards the callback, and `compare_lines_with_lap_model` adds the phase (baseline, full, half, three_quarter fallback, or adaptive fourth) and exact trial track for AI runs. The normal core API pays only a conditional branch when no observer is supplied; callback tests compare observed values with recorded telemetry and unchanged final lap time/energy.

The snapshot also copies the accepted cell's next-entry braking ceiling, motor torque request, front/rear brake pressure requests, achieved drive, friction-brake, and regenerative-brake forces, longitudinal acceleration, and signed pack-terminal power. It reads already solved scalar state; it does not evaluate forces or rerun the vehicle. The progress regression compares all nine values with same-cell saved telemetry and checks that attaching the observer does not change lap time or energy.

The desktop's worker posts no more than about one update every 0.1 s of computation, plus a phase's final accepted cell, to a one-slot queue. The Tk thread consumes only the latest event; it never reads mutable vehicle state. Before the path-constraint prepass finishes, the view says it is preparing path and speed limits. During a solve it shows the accepted endpoint on that trial's **solver-grid reference x/y path**, its modeled speed and lateral acceleration, and a station-based progress bar. Replay buttons and scrubbing stay disabled until a completed run arrives. A stopped run leaves the last accepted step visibly labeled. The display runs at computation speed; it does not pause the solver to animate a real-time driver.

`ui/driver_view.py::DriverPlayback` requires four equal-length, finite, synchronized telemetry arrays: `vehicle.time_s`, `vehicle.distance_m`, `vehicle.speed_mps`, and `vehicle.lateral_acceleration_mps2`. It rejects nonincreasing time, decreasing distance, negative speed, and a station beyond the displayed path. Endurance telemetry begins after an accepted cell, so playback prepends a time-zero display frame at station zero when necessary. Its initial speed is recovered from the first cell's distance, exit speed, and elapsed time. Given a requested display time within a consistent cell, it interpolates speed and station with the same constant-acceleration relation as `Vehicle.update_state`; inconsistent imported cells use linear station interpolation instead. Solver-aligned lateral acceleration holds the **active cell's** solved value because the core produces one force result per accepted cell. Legacy traces without that alignment retain linear display interpolation. It then interpolates x/y by station on the run's solver-grid reference path and computes a displayed heading from nearby path points. The canvas transforms a short path segment into the car-fixed view; the triangle stays fixed while the map line rotates. The GUI uses wall-clock time only to advance the display cursor, with selectable 0.25× to 4× speed. Playback can be scrubbed and zoomed without rerunning physics. After a completed A/B car comparison, **Replay lap** selects either recorded profile and its shared saved solver grid. After completed AI trials, it selects among the geometric baseline, full, half, and fourth trials that completed, each paired with its exact processed path; an audit-failed run is explicitly labeled **diagnostic** in the menu. Switching resets playback to the beginning, retains the previous play/pause state, and makes no new solver call. The selector is disabled if fewer than two completed runs are available.

The monochrome **cell model values** panel has these exact sources and display conversions. Each quantity belongs to the accepted cell; it is a command or achieved model value as labeled, not a measured driver action. The next-entry ceiling is `constraints.braking_speed_ceiling_mps[next_cell_index]`, recorded under `endurance.path_speed_ceiling_mps`; the torque/brake controller also checks the current cell's local corner limit, so this one ceiling is not the whole target-speed rule.

Box	Saved telemetry channel	Display conversion
Next-entry ceiling	endurance.path_speed_ceiling_mps	m/s × 3.6 → km/h
Motor request	controls.motor_torque_request_nm	N·m unchanged
Front brake / rear brake	controls.front_brake_pressure_psi / controls.rear_brake_pressure_psi	psi unchanged; requested pressures
Drive force	vehicle.drive_force_n	N ÷ 1,000 → kN; achieved
Friction brake	vehicle.friction_braking_force_n	N ÷ 1,000 → kN; achieved magnitude
Regen brake	vehicle.regenerative_braking_force_n	N ÷ 1,000 → kN; achieved magnitude
Longitudinal	vehicle.longitudinal_acceleration_mps2	m/s ² ÷ 9.80665 → signed g
Net battery	battery.power_w	W ÷ 1,000 → signed kW; positive discharge, negative charge

For replay, DriverPlayback uses an active-cell zero-order hold on those optional channels **only when** saved station, time, and speed match the solver's cell grid and constant-acceleration distance relation. At an exact interior cell boundary it switches to the next cell; at the final endpoint it holds the last. Missing, nonfinite, wrong-length, or physically invalid optional channels display – independently; unaligned legacy traces show – for all cell decisions. Live progress instead shows the latest accepted snapshot directly, since a one-snapshot preview does not contain the prefix of earlier cells needed for playback alignment. Tk may coalesce intermediate updates, but no extra physics pass is performed.

For a centerline lap, including either car in a profile comparison, the displayed path is the exact centerline solver-grid x/y frozen in the saved run; for an AI trial it is that trial's processed solver path. Live progress and completed replay therefore interpolate the same per-run reference geometry. The separate Analysis course plot still shows the source x/y, which can differ after solver resampling. None of these views is the integrated vehicle x/y/heading state. In particular, the fused default track's x/y and curvature disagree numerically even on the saved grid. The view is a useful time/station and speed preview but does **not** show steering error, tire path tracking, cone proximity, a true driver's camera, or interactive driving. The desktop switches to this tab when a calculation starts, shows preparation until a cell is accepted, and starts replay after a completed lap. The Timed sessions tab remains visibly unavailable until a closed-loop session engine, versioned controller contract, full input/state/environment capture, replay tolerance policy, and linked comparison report exist.

10. Four-wheel lab: state, control, tire, and motion equations

src/lapsim/dynamics/planar.py is a separate **ten-state** ordinary differential equation model. Its state is (X,Y,psi,u,v,r,omega_FL,omega_FR,omega_RL,omega_RR): three planar pose values, three body motion values, and four wheel speeds. PlanarControls holds four steering angles, four signed drive torques, four nonnegative friction-brake torque requests, optional four normal-load overrides, and optional external body force/yaw moment. The desktop holds these controls constant throughout each A or B run; it sets equal left/right front steering and zero rear steering. An external controller can provide a different sequence to run_planar_dynamics. The GUI does not simulate a motor, inverter, pack, torque delivery latency, or controller feedback in this mode. A negative drive torque is a signed generating/reverse mechanical request, not an automatically constrained battery-regeneration command.

The lab's default synthetic vehicle uses m=300 kg, Iz=160 kg m², a=b=0.8 m, tf=tr=1.2 m, R=0.2 m, wheel J=0.3 kg m², mu=1.5, Ck=7000 N/slip, and Ca=8000 N/rad. Its default 1 s maneuver starts at 12 m/s, front steer 1 degree, output step 0.01 s, and rear requests A (120,120) N m, B (110,130) N m; front torques are zero. These numbers are **test scenario inputs**, not a Prius or TREV calibration. They are visible and editable in the lab. PlanarVehicleConfig also holds gravity 9.80665 m/s², regularization floor 0.5 m/s, and maximum internal step 0.002 s unless code changes them.

For front axle distance a, rear distance b, and front/rear tracks tf,tr, wheel body positions are (a,+tf/2), (a,-tf/2), (-b,+tr/2), and (-b,-tr/2) in FL, FR, RL, RR order. With no override, static loads are Fz_FL=Fz_FR=mg b/[2(a+b)] and

$Fz_RL = Fz_RR = mg \cdot a / [2(a+b)]$. They do **not** change with acceleration, grade, bank, aero, or a grip patch. Passing `controls.normal_loads_n` is a prescribed load experiment; it is not suspension physics. A zero-load wheel produces zero tire force but its spin equation still evolves.

At wheel center (x_i, y_i) and steering angle δ_i , the code computes contact velocity in the body frame and rotates it into that wheel's frame:

```
Vxb = u - r y_i;          Vyb = v + r x_i
Vxi = cos(delta_i) Vxb + sin(delta_i) Vyb
Vyi = -sin(delta_i) Vxb + cos(delta_i) Vyb
vref = max(abs(Vxi), v_floor)
slip_i = (R omega_i - Vxi) / vref
alpha_i = atan2(Vyi, vref)
Fx_raw_i = Ck slip_i;      Fy_raw_i = -Ca alpha_i
Fcap_i = mu * road_multiplier_i * Fz_i
scale_i = min(1, Fcap_i / hypot(Fx_raw_i, Fy_raw_i))
Fx_i = scale_i Fx_raw_i;    Fy_i = scale_i Fy_raw_i
```

When the raw force magnitude is zero, $scale_i = 1$. The common scale is a circular **combined** force cap, so large requested longitudinal force reduces the achieved lateral force and vice versa. v_floor is 0.5 m/s in the lab's default synthetic car, preventing division by zero during launch or wheel lock. It changes the low-speed law; it is not a tire measurement. Reverse behavior is qualitative. The grip multiplier changes **only** the force cap; it does not change stiffness, contact load, wheel radius, or the slip formula. A measured tire lookup would need load, slip, camber, road condition, and temperature data with units and validity ranges.

The achieved wheel force is rotated back to body axes. The net yaw moment is $\sum(x_i Fy_i - y_i Fx_i) + M_external$, with positive yaw counterclockwise. A larger forward force at the right rear has a positive yaw contribution. The state derivatives are:

```
Xdot = u cos(psi) - v sin(psi); Ydot = u sin(psi) + v cos(psi)
psidot = r
udot = r v + (sum Fxb_i + Fx_external + Fx_aero)/m
vdot = -r u + (sum Fyb_i + Fy_external + Fy_aero)/m
rdot = (sum(x_i Fyb_i - y_i Fxb_i) + M_external)/Iz
J omega_dot_i = Tdrive_i + Tbrake_applied_i - R Fx_i.
```

Brake torque opposes actual wheel spin. At exactly zero spin it is capped to prevent a requested friction brake from creating wheel rotation in the opposite direction; integration also handles an incipient zero crossing. These brake rules are in the derivative/integrator, not a hydraulic pressure or ABS model. Unlike the main lap, the four wheel inertias are explicit energy and dynamic states, so they are **not** reflected into the body mass.

10.1 Power identity and integration

Every force evaluation computes $E = 0.5m(u^2 + v^2) + 0.5Iz r^2 + 0.5J \sum(\omega_i^2)$. Mechanical drive and brake powers are $\sum(Tdrive_i \omega_i)$ and $\sum(Tbrake_applied_i \omega_i)$. Contact dissipation is $\sum[Fx_i(R \omega_i - Vxi) - Fy_i Vyi]$. External power is $Fx_external u + Fy_external v + M_external r + Fx_aero u + Fy_aero v$. The diagnostic is:

```
power_residual = E_dot - P_drive - P_brake
                + P_contact_dissipation - P_external.
```

It should be at roundoff for a force evaluation, including the added aero work. This identity checks sign and accounting consistency at an instant. It does not prove that an RK4 trajectory conserves energy exactly, that every requested torque is hardware-feasible, or that the tire parameters are real. The code records the diagnostic in `PlanarEvaluation` and shows it at the GUI cursor.

`run_planar_dynamics` uses classical fourth-order Runge-Kutta with held controls/environment over an output interval, splitting that interval into smaller internal steps. The cap is $\min(user_max_step, 0.8/\lambda)$, where $\lambda = (Ck/v_floor)(R^2/J + 4/m)$ approximates the fastest small-slip mode. The synthetic config's explicit maximum is 0.002 s; stiffness may force smaller substeps. The public run stores $N+1$ boundary times/states and N pre-control interval evaluations. There is no final-point

evaluation because no control interval follows the final state; the GUI computes one only for display with the last held control. A single-step API raises on an invalid road query; the batch runner returns a flagged run. The output count and internal substep count have explicit upper bounds in code. The integration is deterministic for identical input values and code, apart from ordinary floating-point/runtime variation.

11. Report II upgrade: wind and road conditions

`src/lapsim/dynamics/conditions.py` defines immutable `PlanarEnvironment`, `PlanarRoad`, `RectangularGripPatch`, and optional `RoadDomain`. Wind is a **world-frame vector** (w_x, w_y). For heading ψ , the derivative first rotates it into the body frame, then subtracts it from ground velocity:

```
w_body = R(psi)^T w_world
U_body = (u,v) - w_body;      Vair = hypot(Ux,Uy)
F_aero_body = -0.5 rho CdA Vair U_body
P_aero = F_aero_body dot (u,v).
```

The final line uses **ground** velocity for the vehicle's mechanical power balance. Replacing it with $F \cdot U$ would account for a different energy transfer involving moving air. A headwind raises apparent speed and drag; a tailwind faster than the car can make the modeled aero force accelerate the car. C_dA is a single, isotropic, synthetic drag area placed at the center of gravity. A crosswind therefore produces a lateral component under this simple vector drag law, but the code does **not** claim a measured side-force coefficient, aero yaw moment, downforce, front/rear balance, CFD map, or ride-height coupling. `drag_area_m2=0` disables aero exactly and is the core API's backward-compatible default. The GUI starts with a clearly synthetic $C_dA=0.8 \text{ m}^2$, zero wind, and $\rho=1.225 \text{ kg/m}^3$; those values are study inputs, not team-car data.

For each wheel center, the code evaluates world position $(X+\cos(\psi)x_i-\sin(\psi)y_i, Y+\sin(\psi)x_i+\cos(\psi)y_i)$ and queries the road there. The base material is `reference_pavement` with multiplier 1. A finite rectangular patch can assign a different nonnegative multiplier and material identifier. Patches are world-fixed, inclusive of their boundaries, and searched in tuple order; the first matching patch owns an overlap. The GUI can turn on one assumed low-grip rectangle and shows its top-down outline. It passes the **same** environment object to A and B. The code makes no claim that the shipped endurance course has this road patch.

`RoadDomain` is a programmatic optional rectangle representing where road data are supported. If a wheel query leaves it, the road uses the base grip **as a diagnostic fallback** and sets `valid=False`; the result must not be ranked as a validated comparison. The batch run's `road_valid` flag aggregates all wheel force evaluations, including intermediate RK4 stages and the final state. `invalid_road_queries` counts invalid wheel-force queries, including repeated substep evaluations; it is neither elapsed time nor a count of unique places. The GUI reads the run-level flag, labels the result `INVALID ROAD DOMAIN`, and withholds the B-minus-A ranking. The GUI currently does not expose a domain editor; an API caller can supply one.

11.1 What Report II does and does not authorize

Report II is a design and verification source, not a command to copy its synthetic values into the team car. Its first defensible change is to keep the lap solver's established flat baseline while adding explicit air-relative wind and road contact queries to the independent-wheel lab. The report recommends a progression from scalar coefficients and flat road through measured six-component aero maps, surveyed elevation/bank/curvature, dynamic suspension/contact, and conservative thermal states. This implementation stops at the first environment layer because no reviewed team CFD map, ride-height schedule, tire/road friction map, elevation/bank survey, cell thermal characterization, or cooling test is available here.

The main distance-domain lap still uses ground speed as airspeed, a constant density, scalar drag/downforce, and flat uniform road. The new `PlanarEnvironment` is **not** passed into `run_one_lap`, `PathConstraintSolver`, or `Vehicle.update_state`. It would be incorrect to claim that a wind or low-grip value entered in the lab changes lap time. Connecting them safely requires a path-referenced world wind field, force/moment allocation without double counting, a contact-point road query compatible with the lap model, and new regression/validation cases. Sections 16-17 define the evidence gate for that work.

12. Four-wheel GUI inputs and outputs

`ui/dynamics_lab.py::ManeuverSettings` freezes a car config, initial speed, steering, duration, output step, four A torques, four B torques, equal-total requirement, and one shared environment. It validates finite values, speed 0-40 m/s, steer within 30 degrees, duration $(0, 10]$ s, output step 0.002-0.1 s, integral 1-2000 output steps, and per-wheel torque magnitude ≤ 500 N m. Equal total torque is on by default so an A/B yaw comparison isolates left/right allocation more clearly; it is a **sum of torque requests**, not a guarantee of equal electrical power or equal tire force. `run_maneuver_pair` gives A and B identical initial state, front steer, duration, output times, synthetic car, and environment. Initial wheel speeds are u_0/R .

The side panel exposes the car's mass, yaw inertia, geometry, wheel radius/inertia, tire μ , and two stiffnesses. Each entry is an explicit synthetic parameter. World X/Y wind, air density, drag area, base grip scale, and optional patch rectangle/grip scale appear in simple number boxes. A/B wheel torques appear in a four-row table. The plots show top-down paths, yaw-rate histories, four wheel-slip histories for the selected car, and a body-frame wheel diagram. The time slider changes only inspection. Numeric boxes show time, body forward/lateral speed, heading, yaw rate, yaw moment, peak tire use, instantaneous power residual, apparent airspeed, aero body X/Y forces, and whole-run road coverage. The coverage box says ASSUMED when no validity domain was supplied; otherwise it reflects the entire run, not only the cursor sample. The wheel diagram shows each tire's slip, local longitudinal/lateral force, normal load, and road grip multiplier. Dark mode reverses black/white surfaces; it never changes physics.

The path is a computed open-loop **maneuver trajectory**, not the 989 m Formula SAE course. The main app's top-down course is a reference map. Neither view uses 3D animation, elevation, or moving-camera effects. A/B final yaw differences are reported only for domain-valid runs and must be interpreted as sensitivity of this synthetic model, not a control-system lap-time gain. The selected scenario and time cursor choose an evaluation already generated from the frozen settings; editing a text box does not retroactively change a displayed run until Compare A and B is pressed.

13. Profile, source-data, and model audit trail

The profile selector's name is a label; the **effective constructed vehicle** is the physics input. `profiles/registry.py` identifies built-ins and optional locally reviewed TREV source bundles. `profiles/adapters.py` maps allowlisted source values into a fresh `Vehicle`, records inherited defaults and unused fields, and returns a `ResolvedManifest`. The local editable garage in `ui/garage.py` is a small seven-field Prius-equivalent convenience layer. It should not be mistaken for a full calibration manager. A software lead can trace a lap number by reading, in order, the selected profile and overrides, the resolved manifest, the actual effective vehicle snapshot, the resampled track grid, solver settings, controller fraction, result status, and telemetry channels in its saved record.

Data and code are separated deliberately. The shipped course CSV and its metadata are historical inputs; `docs/model_parameters.md` and validation notes describe where baseline values came from; the two ENME408 PDFs are research/design evidence; local Downloads source bundles remain optional and are never silently invented. A source value is only promoted when adapter logic accepts its field and units. A missing 2026-27 parameter remains an inherited assumption or an explicit unknown. The code does not infer a two-motor topology, battery thermal parameters, CFD coefficients, tire temperature law, track bank, or road friction from the research prose.

The important reproducibility distinction is **model input versus measured evidence**. A JSON record's hashed model configuration and track grid are enough to inspect what that software run attempted. They are not a signed calibration certificate, a raw log archive, or proof that the course and car matched an event. The detailed run-record schema and replay gaps are in Section 14.

14. Saved records: exact schema, integrity, and replay limits

14.1 Main-lap record (`experiments/run_record.py`)

`LapRunSettings.from_track` freezes the **resampled solver grid**, not just a filename: `closed`, all distance/X/Y/curvature values, and a SHA-256 of canonical JSON for that geometry. It also saves course ID, length, cell count, requested maximum step, path-constraint settings, constant driver torque fraction, one-lap event settings, and selected profile identifiers. The course ID distinguishes ordinary fused-team runs (`team_endurance_fused_gnss_imu`) from synthetic-loop runs (`synthetic_rounded_rectangle_v1`) rather than silently treating them as the same input. `capture_lap_run` combines these settings with the source-aware profile manifest, editable overrides, full effective `Vehicle` constructor-field snapshot and its hash, result status/reason/time/pack energy/final SOC, and synchronized post-cell telemetry. Each telemetry channel has values, unit when identified, origin category, sample count, and missing-sample status; a missing numeric sample is JSON `null`, not an undocumented zero. The record carries Python implementation/version, operating-system descriptors, and installed NumPy/SciPy/Matplotlib versions. `schema_version=2` embeds the grid; the loader accepts the older v1 format too.

Desktop records also freeze `settings.track.source_course`, independent of `settings.track.geometry_sha256`. It records selected course ID and source geometry hash for every ordinary, A/B, and AI trial. A versioned import adds `revision`, `bundle_id`, canonical manifest `bundle_sha256`, declared raw source hash, coordinate-frame declaration, boundary status, and the exact loaded bundle-file hash. A local reserialization may change that last raw file hash while preserving the canonical manifest hash. Built-in legacy and synthetic courses instead record their respective source CSV/sidecar hashes or generator revision and use `null` for bundle hash. These are frozen provenance claims and integrity links, not proof of survey accuracy. The source hash and solver-grid hash can legitimately differ after arc subdivision or legacy resampling.

For an AI-mode desktop run, `settings.path_planning` records

`algorithm=periodic_cubic_minimum_curvature_slsqp_v6_clearance_probe`, `fourth_strength_policy=eligible_quadratic_or_clearance_probe_v2_fallback_0.75`, `rank_status`, eligible versus diagnostic times, the sampled path audits, and whether the primary record is diagnostic only. It also stores the source-course ID, label, description, and `synthetic_course` flag; source, processed-baseline, and selected-solver geometry audits; a SHA-256 of the **source** geometry; requested centerline step; actual planner spacing/controls/smoothing/iteration cap/length penalty; the assumed corridor source and dimensions; each tested candidate offset strength/path length/time/error; best candidate and selected offset strength if one exists; maximum offset and constraint violation; maximum normal-coordinate fold ratio; source closure/preparation discrepancy; and measured planner/lap-comparison compute time. The speed-only seam policy, 0.005 m/s tolerance, and two-pass limit per trial are explicit. The embedded `settings.track.geometry` is the **primary displayed solver path** and its hash, so a reviewer can inspect exactly what the saved lap used. The saved endurance settings contain the final pass's explicit starting speed; the `result` includes actual entry speed, exit speed, and finish-minus-start seam-speed difference. A default run has no `path_planning` field. Optional planning metadata is validated as canonical finite JSON at record construction, and the record's content ID covers it. The AI workflow saves the primary displayed run and every other completed baseline/full/half/fourth trial, for at most four path records per comparison. Each completed trial file contains its own exact solver geometry, full result, aligned telemetry, and explicit final-pass start speed. The primary file's `comparison_counterpart_run_id` and `comparison_counterpart_role` still identify the comparison counterpart when one exists. Its `record_role` is `selected_result` for the displayed result even when `diagnostic_only=true` and **no racing-line winner was selected**. That diagnostic flag also applies when a candidate completed but the eligible geometric baseline failed; showing that candidate is not an A/B time gain. Other linked files have `record_role=comparison_counterpart` or `candidate_trial`; their `comparison_role` and `offset_strength` identify the geometric baseline or candidate path. The primary file also has a compact `baseline_record` and `candidate_trials[]` manifest: saved trials have `run_id` links, while the path in the primary file has `record_role=selected_result` and no self-referential ID. Incomplete extra trials keep their summaries without a replayable file. Read `settings.path_planning.candidate_trials[].offset_strength` with its sampled audit and eligible or diagnostic time, plus `selected_offset_strength` and `rank_status`; the fourth trial's list position does not imply strength 0.75 or a valid gain. The primary manifest links the available path records, but there is no interactive ghost/session workflow. Re-running the planner requires the source file identified by the hash, equivalent environment/code, and the original vehicle inputs. An individual completed record can be checked through the programmatic model replay in Section 14.2, but stored diagnostic times must not be ranked as racing-line gains.

For AI-generated geometry, `path_planning.actual_maximum_cell_length_m` records the measured largest cell. The legacy solver field `requested_maximum_cell_length_m` receives that measured value for schema compatibility; `path_planning.planner_sample_spacing_m` separately records the nominal 2 m planner grid. The exact embedded cell distances, not either scalar, are the authoritative solver input.

The complete payload is serialized as stable, finite canonical JSON and SHA-256 is its content-derived `run_id`. `RunRecord.save` writes a temporary file and atomically replaces `%LOCALAPPDATA%\LapSim\runs\<run_id>.json`. `RunRecord.load` checks the ID and, for v2, validates embedded geometry, metadata, and geometry hash. Equal content under the same runtime/source identity gets the same name and overwrites that same content; the ID is not a per-attempt timestamp. A failed event result is saved when it reaches capture, but an exception before capture/save leaves no record. The GUI's A/B lap comparison creates two individual run files, not a third linked comparison manifest. Keep the two full IDs together when reviewing a comparison.

For a failed lap, the existing `result.driving_time_s`, `ending_speed_mps`, and `final_state_of_charge` remain the **attempted vehicle state** at termination. A path/time/stall rejection may happen after `Vehicle.update_state` has advanced a cell, while net energy, telemetry, and Driver progress contain only the checked prefix. New `accepted_time_s`, `accepted_distance_m`, `accepted_speed_mps`, and `accepted_state_of_charge` identify that prefix even when telemetry was disabled. `failed_lap_index` and `failed_cell_index` locate the failing cell with zero-based indices; `failed_cell_update_completed` says whether `Vehicle.update_state` returned successfully. A false value can mean an entry-gate rejection or an update exception that partially changed component state. All seven fields are optional additions to the existing v2 result; older manually constructed results and records may lack them. Capture validates a supplied prefix against the last saved telemetry sample and adds a warning when the attempted time is ahead of it. This is **transparent failure accounting**, not a rollback: the caller's vehicle can still contain the attempted component state. A zero-SOC endpoint passes the physical-cell checks, so it is included consistently in energy, telemetry, and progress before the run reports battery depletion.

The embedded grid makes the solver input auditable even if the checkout course file later changes. It does not embed raw GNSS/IMU logs, original fusion scripts, or a signed track survey. The manifest records Git commit and dirty-worktree flag; it does not store the uncommitted patch, so a dirty run cannot be exactly reproduced from the commit alone. Dependency versions are selected key packages, not a complete lockfile or BLAS build. The replay checker below restores only its allowlisted built-in model classes; the snapshot is not a universal constructor for external components. A content hash detects accidental mutation but does not authenticate who supplied the data or prove model agreement.

The AI trial manifest has `trial_record_manifest_version=1` within the existing v2 lap-record schema. Each linked `run_id` is a content hash, but loading the primary file does not load every linked file or make the group an atomic archive. Resolve each saved ID to its own JSON file and load it independently when auditing all paths.

An auxiliary AI path record identifies its own `eligible_lap_time_s`, `diagnostic_lap_time_s`, `trial_error`, `sampled_audit`, and `diagnostic_only` status. Its `comparison_rank_status` is the **overall** four-path decision, not that file's individual result. `comparable_with_baseline` is false when the geometric baseline lacks an eligible time, even if this individual candidate completed. The Driver view's best-path replay label includes the selected offset strength; a baseline-failed candidate and every other ineligible path are labeled diagnostic. These fields prevent a standalone trial file or replay choice from being mistaken for a ranked winner.

14.2 Programmatic one-lap model replay (experiments/lap_replay.py)

`replay_lap_record(path, tolerances=LapReplayTolerances())` accepts one completed, one-lap **v2** JSON record with a complete accepted-cell trace, including any completed AI trial flagged diagnostic. `RunRecord.load` first checks its content ID and embedded track hash. The checker validates the effective vehicle snapshot hash and reconstructs only known model dataclasses from an explicit class allowlist; it does not dynamically import class names from JSON. It restores the exact saved solver grid, path-constraint settings, endurance settings (including an AI final pass's explicit start speed), and six per-cell controls: steering angle, front/rear brake pressure, motor torque request, and front/rear regenerative brake-force request. It requires one accepted sample per grid cell, ordered by increasing time and aligned with the saved cell and lap indices and station boundaries. It then resolves the path constraints and replays those controls through `EnduranceSimulator` in the installed code. This is a replay of the saved modeled lap, not a rerun of AI geometry optimization, sampled corridor eligibility, or a driver's steering policy.

The `LapReplayReport` separately states `model_agreement`, `completion`, `saved/replayed sample counts`, named per-output maximum absolute errors and limits, `mismatch_reasons`, and `provenance_warnings`. It checks summary driving/lap time, start/end speed, net pack energy and final SOC; the aligned sample time, station, speed, and next path-speed ceiling traces; and exact agreement of the six recorded control traces. Default absolute limits are **0.01 s** for time, **0.01 m/s** for speed, **1e-5 kWh** for energy, **1e-5** for SOC, and **1e-8 m** for station. The limits can be supplied explicitly with `LapReplayTolerances`; they measure numerical agreement, not prediction accuracy. The checker does not compare every component telemetry channel. An incomplete/misaligned record, unsupported model snapshot, missing explicit AI start speed, or v1 schema is rejected; a modeled rerun that stops or disagrees returns a structured failed report. It compares saved/current source commit, dirty flags, Python/platform, and selected dependency versions as provenance warnings independently of numerical agreement. Exact agreement on a dirty checkout is still numerically possible, while a clean but different code version can disagree. This operation requires a Python environment with the model and may take a full lap solve; the desktop has no replay-check button.

```
from lapsim.experiments import (
    LapReplayTolerances, default_run_directory, replay_lap_record,
)

path = default_run_directory() / "<run_id>.json"
report = replay_lap_record(path, tolerances=LapReplayTolerances())
print(report.model_agreement, report.mismatch_reasons)
print(report.provenance_warnings)
for metric in report.metrics:
    print(metric.name, metric.maximum_absolute_error, metric.tolerance, metric.passed)
```

The file hash checks that the loaded JSON has not changed since it was named; it is **not** this replay check. A successful command replay confirms selected numerical outputs of the saved **model** run and reruns the event's requested/achieved curvature gate. It does not prove that the integrated vehicle x/y followed the separately plotted course or stayed within surveyed boundaries. Neither this API nor the Driver view's visual telemetry playback proves agreement with an instrumented vehicle or implements the disabled Timed sessions workflow.

14.3 Four-wheel A/B record (experiments/dynamics_record.py)

`capture_dynamics_comparison` takes the exact `PlanarVehicleConfig`, `PlanarEnvironment` (including wind, density, CdA, base road, domain, and ordered patches), initial state, every A/B control step, output step, and both `PlanarRun` objects. It checks both runs share the initial state/time grid, that equal-total torque holds at each step if requested, and then **replays both runs** under those frozen inputs before accepting their trajectories or RK-stage road status. The record stores **every** boundary time/state, every pre-step wheel/force/energy evaluation, per-sample body longitudinal/lateral force residuals, yaw-moment residuals, power residuals and their maxima, road-validity flags and invalid-query counts. It identifies the FL/FR/RL/RR wheel order, Python/platform/dependency versions, source commit/dirty flag, and synthetic evidence status.

`DynamicsComparisonRecord.load` checks its content ID, alignment/time grid, torque condition, equation residual summaries, and deterministic replay against the saved input. Both A and B are in one linked file at `%LOCALAPPDATA%\LapSim\dynamics_runs\<run_id>.json`. The UI saves it before displaying a completed comparison and shows the shortened ID.

The lab record supports inspection and programmatic replay from exact inputs, but there is no one-click replay button yet. Its power/force residuals are evaluated from the saved numerical derivatives and forces; they do not compare a simulated maneuver with instrumented car measurements. The road-valid flag reports that the **solver found** every queried contact inside the declared domain; capture and load rerun the integrator to check this, including RK4 stages. The file still stores only their aggregate invalid-query count, not every intermediate contact coordinate. If no domain exists, coverage is an unbounded synthetic assumption, which the GUI calls `ASSUMED`; it is not a claim of road survey coverage. A/B record capture does not include the current uncommitted source patch. Because load uses the installed model code for replay, a future change to physics or integration may reject a historically valid record until an explicit schema/model-version migration or archived-code replay is provided.

14.4 Practical record inspection

Read the JSON in this order: `schema_version` and `simulation_mode`; `runtime` and source identity; `configuration/inputs`; `settings` and solver grid or time step; `result/runs`; `validity`; then `telemetry/evaluations`. A run is usable for a model comparison only if its event succeeded, its selected inputs match the intended comparison, its declared validity checks pass, and its assumed parameters are appropriate for the engineering question. Never rank a failed lap or a road-out-of-domain maneuver by its partial elapsed time or final yaw alone. Preserve the full JSON alongside any plot shared with the team.

15. How Python modules and the desktop interact

Responsibility	Main source	Contract / important side effect
Desktop entry and main widgets	src/lapsim/ui/__main__.py, ui/app.py	Tk event loop; validates form inputs; dispatches worker; renders results on GUI thread
Four-wheel widgets	ui/dynamics_lab.py	Freezes ManeuverSettings, runs matched A/B in worker, saves one record, draws top-down/force views
Shipped course and gridding	ui/simulation.py, courses/spatial_track.py	Reads source CSV and creates path cells; on-demand SpatialTrack.refine preserves existing cell boundaries for resolution checks
Optional path proposal and A/B timing	optimization/racing_line.py	Builds processed geometric baseline and bounded full/half/fourth-offset candidates; screens integrated curvature paths for sampled clearance/closure before ranking; evaluates at most four paths, with at most eight lap-model passes in the desktop speed-seam policy
Car selection and local saved profiles	profiles/registry.py, profiles/adaptor.py, ui/garage.py	Constructs fresh Vehicle and provenance; seven editable garage fields
Spatial speed constraints	solvers/path_constraints.py	Computes local tire-limited and cyclic braking entry ceilings
One-lap event	events/endurance.py	Starts a rolling initial-condition lap, applies driver/automatic brake actions, calls cell update, records accepted telemetry and seam speeds
Main vehicle assembly	vehicle_model/vehicle.py	Couples aero, suspension, tire, drivetrain, brake, battery and commits state after solving a cell
Four-wheel derivative and integrator	dynamics/planar.py	Pure force/derivative evaluation; bounded RK4 time integration; no lap score
Wind/road scenario	dynamics/conditions.py	Deterministic world wind and world-fixed contact-patch query; no update side effect
Main-lap evidence	experiments/run_record.py	Content-hashed v2 record with exact solver grid and effective vehicle snapshot
One-lap model replay	experiments/lap_replay.py	Reconstructs an allowlisted vehicle and saved grid; reruns accepted-cell controls and reports numeric differences separately from provenance
Driver view and progress	events/endurance.py, ui/driver_view.py, ui/app.py	Emits immutable accepted-cell snapshots; maps live endpoints and completed telemetry to each run's saved solver-grid reference x/y; switches among already solved comparison laps without changing physics
Four-wheel evidence	experiments/dynamics_record.py	Linked A/B content-hashed input/trajectory/force record

An implementation change should begin with the **contract** between these modules. For example, adding measured wind to the lap solver is more than another GUI box: the track needs an orientation/world frame, the path solver must use air-relative forces at its speed-ceiling probes, `Vehicle.update_state` must use the same force without double counting, the record must store the wind field and its frame, and analytical baseline/headwind/crosswind checks must pass. Likewise, adding dynamic ride height requires new continuous states/contact geometry in the time-domain model or a defined equilibrium map in the

distance-domain one; a plotted height slider alone would have no physical effect.

15.1 What to change for a new car profile

Create a reviewed parameter table with unit, source, date, vehicle configuration, operating range, and uncertainty. Add an allowlisted adapter mapping, then inspect the resolved manifest and effective vehicle snapshot. Confirm motor count/topology, wheel radii, torque/RPM/power envelope, cell/pack limit behavior, mass/CG/inertia, aero convention, tire load/slip data, braking/regen boundaries, and track condition separately. Do not fill missing fields with a plausible number without labeling it an assumption. The garage's seven boxes are useful for an initial sensitivity run, not enough to assert a 2026-27 calibration. Re-run the baseline unit/equation checks and held-out data comparison after any physical parameter update.

15.2 What to change for a new track

Supply a source-controlled or recorded course with monotone distance, x/y, signed curvature, closure decision, units, and derivation metadata. `resample_track` can create solver cells, but a new course should be checked for geometric closure, map/curvature agreement, and cone/path width. A legitimate Report II road extension additionally needs elevation, grade, bank, crest curvature, surface material/condition, and validity coverage referenced to a shared world frame. Compute gravity along and normal to the road consistently: grade changes required climbing work and normal force; bank changes lateral tire demand; a crest can unload contact. Those are **not** currently in the lap model. A local wetness or grip multiplier without a surface measurement should stay a sensitivity scenario.

For the current desktop import path, derive a coherent closed constant-curvature solver course first, export the four-column `SpatialTrack` CSV, and run `scripts/create_course_bundle.py` with source, frame, revision, processing, and review declarations. The converter and importer enforce internal geometry, hash, size, and signed-turn gates; they do not fit raw measurements or accept measured boundary widths. Select its JSON through **Import course....** Keep the source CSV and preparation method with the generated bundle and run record. Changing any manifest content requires a new revision, and a complete source/solver geometry plus measured-boundary review is still needed before interpreting path clearance physically. See `docs/course_bundle_format.md`.

15.3 Numerical and code-review guardrails

Keep derivative evaluations pure during RK4 and commit thermal, charge, wear, or weather-state evolution only after an accepted step. Keep one definition for each force and sign; aero loads must enter both force balance and power/accounting exactly once. Define whether a coefficient maps body velocity, air-relative velocity, or wheel-contact velocity. If adding pitch/yaw aero moments, verify moment reference points and equilibrium; if adding rotating states, remove any equivalent-mass term representing the same inertia. Preserve dimensional units at API boundaries. Version every new record field/schema and add a migration or explicit rejection path. The deterministic baseline and limiting-case checks in Section 16 are the minimum regression gate.

16. Checks performed, interpretation, and known discrepancies

This cycle added coarse prescribed-path skidpad regressions: a radius-5 m circle split into one or four cells cannot score after accelerating from rest beyond its modeled corner limit, while an ordinary within-limit skidpad still scores. The optional AI regression requires a 0.95-strength sampled-audit pass and a completed 14.556611 s Prius trial; an independent fixed-path 1 m probe retains roughly 0.449 s over the old 0.75 fallback. The Windows preflight also draws a TkAgg canvas and checks the shipped course before launch.

On 6 October 2026, complete repository coverage across three isolated pytest processes reported **425 passed, 642 subtests, and zero skips**. Splitting the 43 test files avoided a host-wide Windows commit-memory limit; each process used one OpenBLAS, OMP, and MKL thread and explicit Tcl/Tk library paths. The Driver view checks compare every live accepted-cell value with same-cell telemetry, verify replay's active-cell boundary timing and independent missing-channel handling, and confirm the desktop numbers and unit conversions. Failed-run checks cover rejection before a cell update, rejection after the first and a later attempted update, last accepted telemetry alignment, complete-run equality, record

serialization and contradictions, and consistent battery-depletion progress. A/B tests use cars with markedly different independent first-cell ceilings, require one common start and one preparation per car, replay both saved records, inspect live/replay/popup behavior, and reject a different vehicle or track when reusing limits. At the declared minimum **1080 × 720** window, the decision grid requests **620 px** inside a **660 px** panel and the course canvas remains **264 px** high. Course checks include exact catalog-source identity for the synthetic ID, analytic per-cell endpoint and closed-heading validation, original-station-preserving subarcs, and a valid coarse two-semicircle loop whose polygon chord winding aliases the true turn. They also cover bounded versioned JSON and CSV loading, duplicate/nonfinite/malformed input rejection, source and manifest hashes, local catalog reload and revision conflicts, imported centerline and AI runs, linked trial records, and model replay of those records. These are numerical and software checks, not a track survey or measured-boundary validation.

The repository tests exercise constructors, component limits, path constraints, endurance events, desktop settings, record integrity, and the four-wheel equations. The environment checks cover backward-compatible zero-drag/still-air behavior, headwind/tailwind/crosswind signs and power, rotated world wind, road queries after heading rotation, malformed inputs, deterministic repeated runs, and road-domain violations that occur **only inside RK4 stages**. The four-wheel lab checks that A/B share initial conditions and the same wind/patch, and that its worker saves a linked record. The record checks cover content hashes, geometry/input alignment, schema compatibility, tamper rejection, and residual summaries. The aero/suspension guards reject unsupported loss of normal reaction. Course-catalog checks cover source identity, synthetic and imported arc geometry at several requested steps, exact per-source-cell grid limits, fused-grid behavior, and invalid grid requests; desktop checks cover course-switch clearing, saved course IDs, import persistence, and comparison provenance. The exact command, result, and Git commit should be logged together for a later design review.

Focused planner regression checks exercise explicit corridor requirements, x/y-derived rather than copied curvature, exact chord/station reconstruction, periodic geometry and winding, deterministic bounded iteration, variable widths, narrow 0.5 m source cells, self-crossing and folded-corridor rejection, full/half/adaptive-fourth-offset trials and fallback, failure handling, and candidate selection with the full lap model. The narrow-cell QA case permits only about ± 0.01 m offset; its reported candidate offset was about -0.00916 m with zero computed bound violation. A historical **one-pass** synthetic rounded-rectangle case with the built-in Prius at fixed 0.8 torque fraction gave a 16.784 s processed baseline and 14.267 s full-offset candidate; those are not current AI desktop times. A sampled circle regression checks near-uniform curvature and exact integrated turn of 2π after resampling. Separate `SpatialTrack.geometry_audit()` tests catch map chords longer than their station cells and accept shorter geometric chords around curves; new tests cover source turn mismatch, curvature-integrated closure, and individual arc-chord mismatch when whole-lap totals happen to agree. `SpatialTrack.refine` tests retain original station/curvature boundaries, signed turn, and squared-curvature integral while bounding pathological grid size. New planner tests check sampled modeled-path clearance and closure, preserve completed but ineligible diagnostics, and prohibit ranking when the processed baseline fails. Driver-view checks include constant-acceleration interpolation, inconsistent legacy fallback, closed-course seam wrapping, and switching already solved A/B and AI paths, including diagnostic replays. Progress callback tests compare snapshots against telemetry, verify unchanged lap time/energy, suppress rejected-cell updates, and exercise phase/track forwarding and desktop queue coalescing. Lap replay tests cover completed centerline and speed-periodic records, changed commands/results, missing channels, misaligned samples, unsupported model snapshots, off-path steering, and incomplete or legacy records. Current-cell corner-exit tests exercise both automatic capping and explicit-control rejection; curvature-gate tests reject direct zero-steer circular driving, lateral-force saturation, and a small persistent steering bias. These tests establish model/software behavior, not surveyed Formula SAE feasibility.

Additional targeted regressions place a narrow source corridor cell entirely between prior quarter-cell audit samples and require the updated audit to reject the modeled arc. They check that the closed seam applies the narrower of the first and last supplied widths, that the synthetic grid cap counts generated cells per segment, and that every available runnable profile can enter the same optional AI comparison path without a profile-specific planner branch. These checks establish implementation behavior on synthetic inputs; they do not validate a measured boundary or any vehicle dynamics.

The following **speed-only seam shooting** calculations used torque fraction 1.0, a fresh fixed initial car/pack state per pass, one dry plus one recorded pass per path, and a 0.005 m/s speed residual tolerance. They were first benchmarked **before the sampled modeled-path gate** and are now retained as completed diagnostics rather than eligible racing-line comparisons. With the built-in Prius and a **user-assumed** ± 2 m half-width, 1.78308 m car width, and 0.3 m margin, the processed x/y geometric baseline was **1,011.080 m** long after at most **0.769 m** preparation shift, versus 989 m of source stations.

Baseline/full/half completed at **87.618366/87.377773/87.403816 s**; the earlier policy selected full on its **0.240593 s** numerical lead. At assumed ± 2.5 m, baseline/full/half completed at **87.618366/87.441030/87.375223 s**; the old policy selected half on a **0.243143 s** numerical lead. These prior selection labels are historical, not current audit eligibility. An ordinary 2 m source-curvature Prius run completed in about **78.787 s**, but its 989 m station path is different geometry and uses the default one-pass start policy. It must **not** be subtracted from an AI time as a racing-line gain. Earlier one-pass AI times of 87.364 s baseline and 87.155 s full Prius ± 2 m predate speed-only seam shooting and the current-cell corner-speed guard. The team's widths, car limits, and track conditions are not established by these scenarios.

The **current sampled audit** rejects all four assumed ± 2 m Prius paths under the **1.78308 m vehicle width and 0.3 m margin** used for this benchmark, which differ from the desktop's initial 1.8 m and 0.2 m fields. Baseline/full/half/three-quarter usable-corridor excess is **0.095506/0.908195/0.501981/0.705117 m**, with integrated position-seam gaps about **0.75–0.77 m**. All four complete the model lap at **87.618366/87.377773/87.403816/87.3655968872 s**, respectively, and the failed baseline audit makes the fourth strength fall back to 0.75, but rank_status=invalid_processed_baseline, eligible baseline and candidate time fields are null, and **no AI winner or gain is selected**. The three-quarter number is the fastest of these diagnostics, not an eligible line-selection result. The desktop marks the completed times with *, blanks the difference, disables **Compare path numbers**, and retains diagnostic replays and linked records. The ordinary default centerline is unchanged.

The **current TREV5 working geometry** run with an assumed ± 2 m half-width, 1.8 m width, 0.2 m margin, and torque fraction 1.0 also has rank_status=invalid_processed_baseline. Its baseline/full/half/three-quarter completed diagnostic times were **62.1006641048/62.1124557233/62.0913539700/62.0959759770 s**. Sampled usable-corridor excess for full/half/three-quarter was **0.908612549/0.456447888/0.682565568 m**, and their integrated position-seam gaps were **0.771628008/0.761433514/0.766570964 m**; the baseline itself also fails the audit, so the fourth strength falls back to 0.75. The half-strength number is the fastest diagnostic, but no path is eligible for selection. This four-path comparison took about **30.14 s** on this machine, not a latency guarantee.

The optional **synthetic rounded-rectangle course** provides a separate controlled software example. Its ID is synthetic_rounded_rectangle_v1, length **195.398224 m**, generated source-cell limit **0.5 m**, and four radius-12 m quarter-circle arcs joined by two 40 m and two 20 m straights. The desktop's initial AI corridor **± 3 m**, vehicle width **1.8 m**, and safety margin **0.2 m** are assumed. Under the current GUI policy with the built-in Prius and torque fraction **0.8**, the speed-periodic processed baseline completed in **16.8855728415 s**, the half-offset path in **15.6242848232 s**, and the selected 0.95-offset path in **14.5566114677 s**; all three passed their sampled modeled-path audits. The full-offset path completed in **14.4579210447 s** but failed the sampled corridor by **0.0406357247 m**, so its faster time is diagnostic. The audit-valid 0.975 trial had only **0.006407 m** of sampled slack, below the extra 0.02 m selection buffer. The 0.95 path had **0.053411 m** and became the fourth physics trial. rank_status=candidate_selected applies to that path with a **2.3289613738 s** modeled lead. This proves the optional AI comparison can produce an eligible selection on this analytic shape. It does not validate a real Formula SAE course, measured cones, Toyota hybrid dynamics, or a Racing Terps car.

An on-demand fixed-path resolution probe used SpatialTrack.refine(1.0) on that synthetic baseline, old 0.75 fallback, and selected 0.95-offset path, preserving original cell boundaries and curvatures. Their refined two-pass laps completed and closed speed: baseline **16.882064565 s**, 0.75 offset **15.000468902 s**, and 0.95 offset **14.551839123 s**. The selected path retained about **0.448630 s** over the old fallback. Refined path clearance was **not re-audited**, so the probe supports only a timing-sensitivity observation on fixed paths, not renewed path eligibility or formal grid convergence. The recorded default course's failed gate remains unresolved.

An earlier on-demand speed-periodic resolution check held each generated assumed ± 2 m path fixed and used SpatialTrack.refine(1.0) to split existing cells **without averaging curvature across their original boundaries**. The original paths had 495 cells each and maximum cell lengths of **2.873–3.368 m**. Refined paths had **1,335–1,344 cells**. Prius baseline/full/half times were **87.567225/87.330961/87.356931 s**; the full path retained a **0.236265 s** numerical lead versus **0.240593 s** on the original grid. TREV baseline/full/half times were **62.013863/62.049208/62.015252 s**; baseline beat half by **0.001388 s**, reversing the original-grid **0.009310 s** half-offset lead. All six refined laps completed and closed seam speed, but no sampled modeled-path clearance/closure audit was applied to those prior trials. A two-pass refined Prius trial took about **9.5 s** and a TREV trial about **20 s** on this machine; this diagnostic is deliberately outside the default run path. Earlier

global-grid resampling also reversed TREV's numerical order but averaged across curvature changes. Neither check is a formal convergence study or measured validation, and none establishes current path eligibility.

An earlier offline **one-pass** resolution probe held the geometric paths fixed while resampling their solver cells from roughly 2 m to 1 m. In the assumed ± 2 m Prius scenario, a 0.75-strength offset beat the full offset by **0.025 s** on the coarse grid, then lost by **0.035 s** on the finer grid. A TREV5 0.25-strength offset similarly changed rank against its geometric baseline. These small differences are below the demonstrated grid sensitivity. The current policy tests 0, 1, and 0.5 first, then makes one fourth trial chosen by eligible quadratic, clearance screen after an invalid full path, or 0.75 fallback. The four-path budget does not make hundredth-second gains robust without a resolution study. This is a numerical sensitivity check under the earlier start policy, not track validation or a formal convergence study.

An independent seam audit used the unprocessed **989 m, 1,978-cell** shipped centerline, the default Prius benchmark, a 0.8 torque request, and no start-speed override. It calculated **79.161 s**, starting at the braking ceiling of **13.2725 m/s** and finishing at **9.4656 m/s**: finish minus start was **-3.8069 m/s** at the same closed-course seam. This is a measured software/model discrepancy for the default one-pass definition, not a claim that the real car loses that speed on every lap. The result and saved record expose both speeds. Optional AI timing now uses the bounded speed-only seam check above; it does not make the changing pack and other component states periodic. Neither route is a verified steady-state lap prediction, and different paths, seams, or start policies could affect ranking.

These are software and equation checks. For an actual car model, additional independent evidence is needed: measured acceleration/coast/braking on specified surface, wheel torque/RPM and pack voltage/current, synchronized yaw/IMU/GNSS, tire force/slip/load and temperature, measured aero balance or CFD with a declared coordinate convention and validation, and surveyed course/ambient conditions. Fit one subset of runs, then test on held-out days, speeds, steering maneuvers, and tracks. Report time error, speed/yaw residuals, energy error, constraint violations, and parameter uncertainty separately. A single matched lap time can hide compensating wrong drag, grip, power, and brake models.

Existing historical material is useful but limited. `docs/model_parameters.md` states that rolling radius, motor efficiency, and chain efficiency were fitted to the same endurance data used for a historical comparison, so that agreement is **in-sample**. `docs/first_lap_soc_validation.md` records 39.07 s simulated versus 69.49 s measured for one first-lap comparison and 13.28 m/s speed RMSE; some discussion there predates later brake logic and should be rechecked before quoting it as current behavior. `docs/real_accel_validation_report.md` documents a rear-wheel-derived acceleration target and explains why it is not a clean independent gate. `docs/battery_rc_validation.md` gives a limited chronological voltage holdout for one historical pack log (about 2.669 V RMSE over its final 35%). None of these constitute a complete 2026-27 Formula SAE vehicle validation, a tire thermal calibration, or a Report II CFD proof.

Important current bounds and gaps:

Boundary	Consequence for interpretation
Main lap follows prescribed curvature and no cone/width rule	A fast time is not proof of a feasible driven line
Explicit-control lap checks requested and achieved curvature, but not x/y path or boundaries	Passing the scalar curvature gate and recorded-control replay does not prove the car followed the displayed map or cleared cones
AI polygon offset check uses a declared synthetic corridor relative to processed x/y	It verifies the proposed spline offset under supplied widths, not the arc path implied by curvature
AI sampled modeled-path check requires clearance and position closure	It screens four positions per modeled cell plus every source-corridor boundary and midpoint, using the narrower adjacent width at shared boundaries; it does not certify continuous swept-body or surveyed cone clearance, and the shipped paths fail
AI objective is local mean-curvature optimization plus bounded line-strength trials	Up to eight full-model passes produce diagnostic runs, but no path is selectable when the processed baseline fails its modeled-path audit

Boundary	Consequence for interpretation
Source x/y and curvature disagree	Stored turn is 3.657937 rad versus 6.283185 rad map winding; default and AI paths cannot be compared for a racing-line gain
Default lap speed need not close at the course seam	The default starts at a cyclic braking ceiling for one pass; AI trials separately require speed closure within 0.005 m/s, but no route enforces periodic charge or other full state
Driver view maps solved station onto reference x/y	The marker is not the dynamically integrated vehicle pose or an interactive driver
Main lap uses flat, uniform, still air	Lab wind/road changes do not change lap time
Main tire is capacity-based with inferred slip	No independent wheel transient or temperature prediction in a lap
Lab uses static Fz and synthetic linear/circular tire	It demonstrates yaw/force coupling, not a calibrated car
Lab aero is vector drag at CG, no downforce/moments	Crosswind sensitivity is qualitative without measured coefficients
Lab road patch changes grip cap only	No roughness, rolling resistance, puddling, hydroplaning, or surface heat
No thermal states in either current route	Continuous torque, brake temperature, tire fade, and pack derating cannot be predicted
Main event can update a cell before rejecting its path/time/stall result	Failed records distinguish attempted vehicle time/speed/SOC from the last checked prefix; caller vehicle remains attempted, so neither is an eligible lap time
No packaged EXE or full dependency lock	Deployment is from a configured source checkout and venv

17. Prioritized engineering work and acceptance evidence

- Freeze the 2026-27 car evidence.** Collect a versioned, reviewed configuration: mass/CG/inertias, wheelbase/tracks/radii, drive topology and torque limits, pack/thermal data, brake/regen limits, tire map, and aero coordinate convention. Keep source, units, validity range, and uncertainty per field. Promote only verified values into adapters; keep unknowns explicit.
- Establish held-out baseline validation.** Instrument a known dry, flat maneuver and course section. Align times and frames; compare speed, yaw, wheel speeds/torques, pack voltage/current, and brake response with error and uncertainty. Require zero-weather/constant-surface regression before each environmental change.
- Repair and survey the recorded course before ranking real racing lines.** Measure both cone boundaries in one world frame with x/y, station, closure, and uncertainty. Reconcile 989 m station/curvature with roughly 1,012 m x/y using retained raw data and versioned processing. Build a path representation whose x/y, station lengths, curvature arcs, and position seam agree without introducing damaging curvature spikes. The optional synthetic loop now passes the sampled modeled-path gate under an assumed corridor; retain it as a software check, then test the full swept vehicle envelope against surveyed boundaries. The unmerged arc prototypes are research evidence, not a finished repair. Until then, the default fused course's AI times are diagnostics only.
- Extend wind and aero only with reviewed maps.** A first lap wind extension needs a world-referenced course and measured/declared wind. A full six-component CFD or wind-tunnel map needs force/moment reference, yaw sign, ride height/roll/pitch axes, interpolation bounds, and crosswind benchmarks. Avoid applying both scalar drag/downforce and the same map forces.
- Add surveyed road geometry and contact states.** Elevation/grade, bank, crest, material, and patch coverage must be sampled at actual wheel contacts. Check analytically that still-flat results recover baseline; constant grade gives

longitudinal gravity force $mg \sin(\theta)$, elevation work $mg \Delta h$, and normal component $mg \cos(\theta)$; symmetric bank changes lateral demand with correct sign; crest unloading cannot produce negative contact loads. Track width and cone feasibility require a path model, not a display outline.

6. **Add conservative thermal states only with calorimetry and cooling evidence.** Tire/brake/motor/inverter/pack temperatures need explicit heat capacities, generated heat, conductive/convective/radiative flows, ambient or coolant boundaries, and state-dependent limits. Internal heat exchanges must cancel in the whole-system balance. Slip and brake work must be assigned once, without double counting pack discharge. A derivative call must not change temperature or SOC.
7. **Build timed sessions only after pose tracking and replay contracts exist.** A versioned vehicle and controller must record controls, states, environment, validity, and timing in one frame. Extend the new cell-curvature gate to a checked integrated pose against one consistent course map and surveyed boundaries before calling an explicit-controls run a driven lap. A ghost is only a time/station comparison until both tracked poses are verified. Extend the existing one-lap command replay to complete session states and environment, define channel tolerances, and link both run IDs, source hashes, and software versions in a comparison manifest. Then make the disabled tab a testable workflow.
8. **Harden distribution and replay.** Package course/profile data or document the source-only requirement. Pin dependencies, extend the current one-lap command replay into an automated, version-aware comparison gate, capture dirty patches or require clean code for comparison evidence, and add a standalone car-comparison A/B manifest beyond the current primary AI trial manifest. Document schema migrations and machine-independent tolerances. The four-wheel record already reruns its A/B inputs; neither record type yet supplies the complete timed-session workflow.

Report II's examples and reference checks provide **test design**, not acceptance evidence for this checkout. Each stage above should have an equation/units check, limiting-case regression, synthetic sensitivity with monotonic expectations where applicable, and an independently measured holdout before it is described as predictive. Until then, label displayed results as model estimates and keep the exact record with any engineering decision.